

MICROSAR vMemAccM

Technical Reference

Version 7.05.00

Authors	vircre, viroto, virsrl, virljs, virbka, vireno, virbmz
Status	Released

Document Information

History

Author	Date	Version	Remarks
vircre	2018-03-16	1.00.00	Initial creation
viroto vircre	2018-04-09	2.00.00	Added critical section description (chapter 3.2). STORY-3380: changes due to support of hardware specific functions and multi instance vMem. Updated chapters 2.10.7, 2.11, 3.1.2, 4.1, 4.2.9 and 4.2.12. Added chapter 3.1.3 describing interfaces of vMem which are specified by vMemAccM.
vircre	2018-09-24	2.01.00	STORYC-4408: Support write and erase burst in vMemAccM. Adapted description of Length parameter in chapter 4.2.5 and 4.2.9. Replace "FOTA" with "Software Download".
viroto	2018-12-18	2.02.00	STORY-9128/ STORYC-7262: vMemAccM shall provide information about currently processed job. (added chapter 4.2.16, extended 4.1 and feature tables)
viroto	2019-01-17	2.03.00	STORY-9427/ STORYC-7272: vMemAccM shall support address area variants. (added chapter 2.6.1 and 4.2.17, updated 4.1)
viroto	2019-02-04	2.04.00	STORY-8665: vMemAccM shall retry a failed write job. STORY-9253: vMemAccM shall retry a failed erase job. (added chapter 2.11)
vircre	2019-02-28	2.05.00	STORY-9456: vMemAccM shall evaluate ECC error results returned by vMem.
viroto	2019-07-03	2.06.00	STORY-12777: vMemAccM shall provide a service to get notified about ECC errors (chapter 2.5, 4.1, 4.2.18)
virsrl	2019-07-26	3.00.00	MWDG-120: vMemAccM shall provide a service to check a specific NV RAM address space to be blank/ erased.
viroto	2019-09-02	3.00.01	MWDG-1884: vMemAccM shall clarify the RAM alignment requirements (added chapter 2.12, adjusted others).
viroto	2019-10-29	3.00.02	ESCAN00102322: vMemAccM_HwSpecificReq accepts data

			size 0 but does not accept data pointer == NULL_PTR (chapter 2.10.7).
viroto	2020-04-07	3.01.00	Reworked chapter 2.3. MWDG-2917: Implement the access synchronization in a multi core system (added 2.13)
viroto	2020-05-14	4.00.00	MWDG-3139: vMemAccM shall cancel an ongoing job according to AUTOSAR (chapter 2.4.4) Improvements within chapter 2.13.2.1.
virljs	2020-09-01	5.00.00	MWDG-3510: vMemAccM shall provide an ECC safe read service (chapters 1.2, 2.3, 2.10.2, 2.16.1, 2.16.1.1 and 4.2.4)
viroto	2020-10-07	5.01.00	MWDG-3291: vMemAccM shall distinguish between failed compare request and unequal data (chapter 2.4.8, 4.1). Some improvements and updates.
virljs	2021-01-28	5.02.00	MWDG-4753: vMemAccM shall synchronize NV RAM access on multi core system via satellites (chapter 2.14) MWDG-4751: vMemAccM shall prevent access to NV RAM via API (chapter 4.2.20) MWDG-4900: Add controller specific information (chapter Other tips) and further configuration tips for the satellite use case (Configuration tips)
virljs	2021-04-22	6.00.00	MWDG-4642: vMemAccM shall initialize and call the MainFunctions of the underlying vMems (chapter 1.2, chapter 2.2, chapter 2.3.1, chapter 4.2.2 and chapter 4.2.10)
virljs	2021-05-26	6.01.00	MWDG-4670: vMemAccM shall provide alternate vMem activation and deactivation services (chapter 2.15, 3.1.1, 4.2.21 and 4.2.22).
virljs	2021-08-09	6.02.00	MWDG-5390: Add a downloadable vMem specific configuration switch (chapter 2.15)
virljs	2021-08-23	6.03.00	MWDG-5111: Add job end notifications (chapter 4.5.1)
virbka	2021-09-13	7.00.00	MWDG-5144: vMemAccM shall optimize job processing (added chapter 2.3.2)
virljs	2021-10-05	7.00.01	ESCAN00109986: vMemAccM_GetState does not set the vMemResult to VMEMACCM_JOB_PENDING though vMem is accessing the NV RAM (modified chapter 4.2.16)
Virljs, vireno	2021-11-24	7.01.00	MWDG-3698: vMemAccM shall set up vMem requests in parallel (Added chapter 2.7 and chapter 5.2.3) MWDG-4687: vMemAccM GetState shall operate on Address Areas

vireno	2022-03-25	7.02.00	MWDG-6700: Implement a minimal synchronous vMemAccM (added chapter 2.17)
vircre	2022-05-27	7.03.00	MWDG-6969: Provide vMemAccM_Cancel API in minimal synchronous mode (added information to chapter 2.17)
virbmz	2022-08-19	7.04.00	MWDG-7128 vMemAccM: Provide service to get address area exclusive memory access (added chapter 4.2.23 and extended chapters 1.2, 2.1, 2.16)
virbmz	2023-05-11	7.04.01	MWDG-8118 vMemAccM: ESCAN00114252 Linking Error - Symbols are mapped to two different sections in memory and added a hint to chapter 2.17.
virbka	2023-11-13	7.05.00	MEMSLP-8368: vMemAccM shall support downloadable external vMem drivers (updated 2.2 and 2.15) Adapted naming from “Multi Partition” to “Multi Binary” (updated 2.13 and 2.14) Added chapter 3.3.

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	Specification of Development Error Tracer	V3.2.0
[2]	Vector	MICROSAR vMem Technical Reference	see delivery

**Caution**

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1	Introduction.....	10
1.1	Architecture Overview	11
1.2	Component Interfaces.....	13
2	Functional Description	14
2.1	Features	14
2.2	Initialization	14
2.3	Main Functions	15
2.3.1	Main Function triggering.....	16
2.3.2	Optimized Job Processing.....	16
2.4	Job Results	16
2.4.1	VMEMACCM_JOB_PENDING.....	16
2.4.2	VMEMACCM_JOB_OK.....	17
2.4.3	VMEMACCM_READ_CORRECTED_ERRORS	17
2.4.4	VMEMACCM_JOB_CANCELED.....	17
2.4.5	VMEMACCM_JOB_FAILED.....	17
2.4.6	VMEMACCM_READ_UNCORRECTABLE_ERRORS	17
2.4.7	VMEMACCM_MEM_NOT_BLANK	17
2.4.8	VMEMACCM_MEM_NOT_EQUAL	17
2.5	Error Notification	17
2.6	Memory Address Mapping	18
2.6.1	Memory address mapping variants	20
2.7	Parallel processing of different asynchronous jobs.....	21
2.8	Priority Based Job Processing	21
2.9	Burst Feature	22
2.10	Partitioning of Asynchronous Jobs	22
2.10.1	Read	23
2.10.2	ReadBlank	23
2.10.3	Write	23
2.10.4	Erase	23
2.10.5	Compare	23
2.10.6	IsBlank.....	23
2.10.7	Hardware Specific Requests	23
2.11	Retries of a failed job	24
2.11.1	Retry a failed burst size request.....	25
2.11.2	Retry configuration.....	25
2.11.3	Supported vMemAccM requests	25
2.12	RAM Buffers	26
2.13	Multi binary synchronization – NV RAM synchronization concept	26

2.13.1	Configuration.....	28
2.13.2	Memory Mapping	29
2.14	Multi binary synchronization – Satellite concept	31
2.14.1	Configuration.....	33
2.15	Activate alternate vMems	35
2.15.1	vMem validation	35
2.16	Error Handling.....	36
2.16.1	Development Error Reporting.....	36
2.17	Minimal synchronous mode	39
3	Integration.....	41
3.1	Scope of Delivery	41
3.1.1	Static Files	41
3.1.2	Dynamic Files	41
3.1.3	Interfaces of the vMem.....	42
3.2	Critical Sections	42
3.3	Handling of non-correctable ECC errors	42
4	API Description.....	43
4.1	Type Definitions	43
4.2	Services provided by vMemAccM	46
4.2.1	vMemAccM_InitMemory.....	46
4.2.2	vMemAccM_Init	46
4.2.3	vMemAccM_Read.....	48
4.2.4	vMemAccM_ReadBlank.....	48
4.2.5	vMemAccM_Write	49
4.2.6	vMemAccM_Erase	50
4.2.7	vMemAccM_Compare.....	51
4.2.8	vMemAccM_IsBlank.....	51
4.2.9	vMemAccM_HwSpecificReq	52
4.2.10	vMemAccM_MainFunction	53
4.2.11	vMemAccM_Cancel	53
4.2.12	vMemAccM_GetJobResult.....	55
4.2.13	vMemAccM_GetMemoryInfo	55
4.2.14	vMemAccM_SetReadOnlyMode	56
4.2.15	vMemAccM_GetProcessedLength	56
4.2.16	vMemAccM_GetState	58
4.2.17	vMemAccM_SetActiveVariant	59
4.2.18	vMemAccM_ErrorNotification	59
4.2.19	vMemAccM_GetVersionInfo.....	60
4.2.20	vMemAccM_RejectNvRamRequests	61

4.2.21	vMemAccM_ActivatevMem	61
4.2.22	vMemAccM_DeactivatevMem	62
4.2.23	vMemAccM Exclusive Memory Access	62
4.3	Services used by vMemAccM	64
4.4	Callback Functions.....	65
4.5	Configurable Interfaces	65
4.5.1	Notifications	65
4.5.2	Callout Functions	66
5	Configuration.....	67
5.1	Configuration Variants.....	67
5.2	Configuration with DaVinci Configurator.....	67
5.2.1	Read Request Length	67
5.2.2	Memory Address Mapping.....	67
5.2.3	Parallel processing.....	68
6	Glossary and Abbreviations	69
6.1	Glossary	69
6.2	Abbreviations	69
7	Contact.....	71

Illustrations

Figure 1	AUTOSAR 4.x Architecture Overview	11
Figure 2	Component Overview of one most likely use case	12
Figure 3	Interfaces to adjacent modules of the vMemAccM	13
Figure 4	Memory address mapping	20
Figure 5	Multi binary synchronization architecture example (2 binaries)	27
Figure 6	Multi binary satellite architecture example (2 binaries)	32
Figure 7	Configuration of Fls_30_vMemAccM	34

Tables

Table 2-1	Provided Features	14
Table 2-2	vMemAccM and vMem result combinatorics in case of Error Notification	18
Table 2-3	Service IDs	37
Table 2-4	Errors reported to DET	37
Table 2-5	Development Error Reporting: Assignment of checks to services	39
Table 3-1	Static files	41
Table 3-2	Generated files	42
Table 4-1	Type definitions	44
Table 4-2	vMemAccM_MemoryInfoType	45
Table 4-3	vMemAccM_StateType	46
Table 4-4	vMemAccM_InitMemory	46
Table 4-5	vMemAccM_Init	47
Table 4-6	vMemAccM_Read	48
Table 4-7	vMemAccM_ReadBlank	49
Table 4-8	vMemAccM_Write	50
Table 4-9	vMemAccM_Erase	50
Table 4-10	vMemAccM_Compare	51
Table 4-11	vMemAccM_IsBlank	52
Table 4-12	vMemAccM_HwSpecificReq	53
Table 4-13	vMemAccM_MainFunction	53
Table 4-14	vMemAccM_Cancel	54
Table 4-15	vMemAccM_GetJobResult	55
Table 4-16	vMemAccM_GetMemoryInfo	56
Table 4-17	vMemAccM_SetReadOnlyMode	56
Table 4-18	vMemAccM_GetProcessedLength	57
Table 4-19	vMemAccM_GetState	58
Table 4-20	vMemAccM_SetActiveVariant	59
Table 4-21	vMemAccM_ErrorNotification	60
Table 4-22	vMemAccM_GetVersionInfo	60
Table 4-23	vMemAccM_RejectNvRamRequests	61
Table 4-24	vMemAccM_ActivatevMem	62
Table 4-25	vMemAccM_DeactivatevMem	62
Table 4-26	vMemAccM_SetExclusiveMemoryAccess	64
Table 4-27	vMemAccM_ResetExclusiveMemoryAccess	64
Table 4-28	Services used by the vMemAccM	65
Table 4-29	JobEndNotification	65
Table 6-1	Glossary	69
Table 6-2	Abbreviations	70

1 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module vMemAccM.

Latest Release:	Analyzed AUTOSAR	R19-11
AUTOSAR Schema Compatibility:		R4.0.3
Supported Configuration Variants:		pre-compile
Vendor ID:	VMEMACCM_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VMEMACCM_MODULE_ID	255 decimal

The vMemAccM (Memory Access Manager) coordinates the NV (nonvolatile) memory access by multiple users in order to avoid conflicts with this shared resource.

The basic functionalities offered by the vMemAccM to access the memory are read, write, erase and compare. Note, that the vMemAccM does not access the memory directly itself, but merely forwards requests to the underlying vMem drivers (Memory Driver, for a documentation of the vMem see [2]).

The vMemAccM was originally created within the new Vector solution for Software Download, which is also its most likely use case. In this case the vMemAccM coordinates the memory access between the classical AUTOSAR memory stack and the Software Download stack. Another likely use case is the memory access by the HSM (Hardware Security Module).

To manage the memory access, the vMemAccM implements the following main features:

- ▶ To avoid clashes within the memory resource, e.g., the flash memory, the vMemAccM processes only a specific number of asynchronous memory access job at a time. See chapter 2.7 for more details on parallel job processing. However, to still be able to accept multiple jobs for different regions of memory the vMemAccM provides a queueing mechanism. Furthermore, the order in which the jobs within the queue are being processed is based on priorities to best meet the requirements of the various users.
- ▶ In order to offer higher layers independence from the actual memory distribution within the underlying vMem modules, the vMemAccM performs a memory address mapping. Upper layers only use virtual addresses to access memory. The virtual memory is structured in so called address areas. The vMemAccM performs the mapping from virtual addresses to pseudo-physical addresses and forwards requests to the underlying vMem modules.
- ▶ To enable higher system response time for higher job priority execution, the vMemAccM divides the asynchronous jobs up into jobs of smaller length, which are

forwarded to the underlying vMem. This way the related vMem is only blocking for a short time, before a different asynchronous job with higher priority could be started. The length of the forwarded job depends on the specific request and configuration.

1.1 Architecture Overview

Note that vMemAccM is not an AUTOSAR module. However, if one tried to place it in the AUTOSAR architecture it would be a complex driver as shown in the following figure.

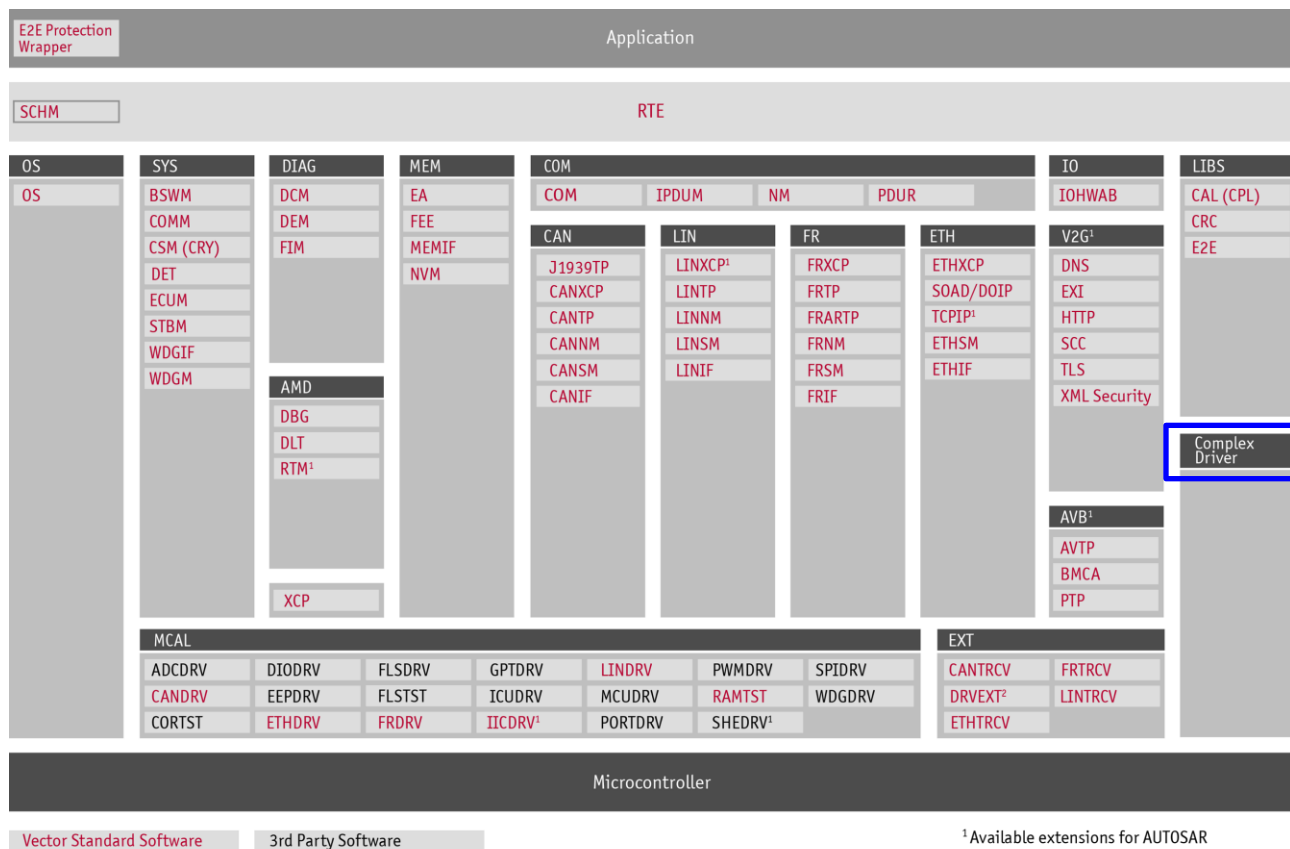


Figure 1 AUTOSAR 4.x Architecture Overview

The following figure shows a possible location of vMemAccM in relation to the AUTOSAR memory stack and the Software Download stack (in the figure referred to as OTA for “Over The Air”).

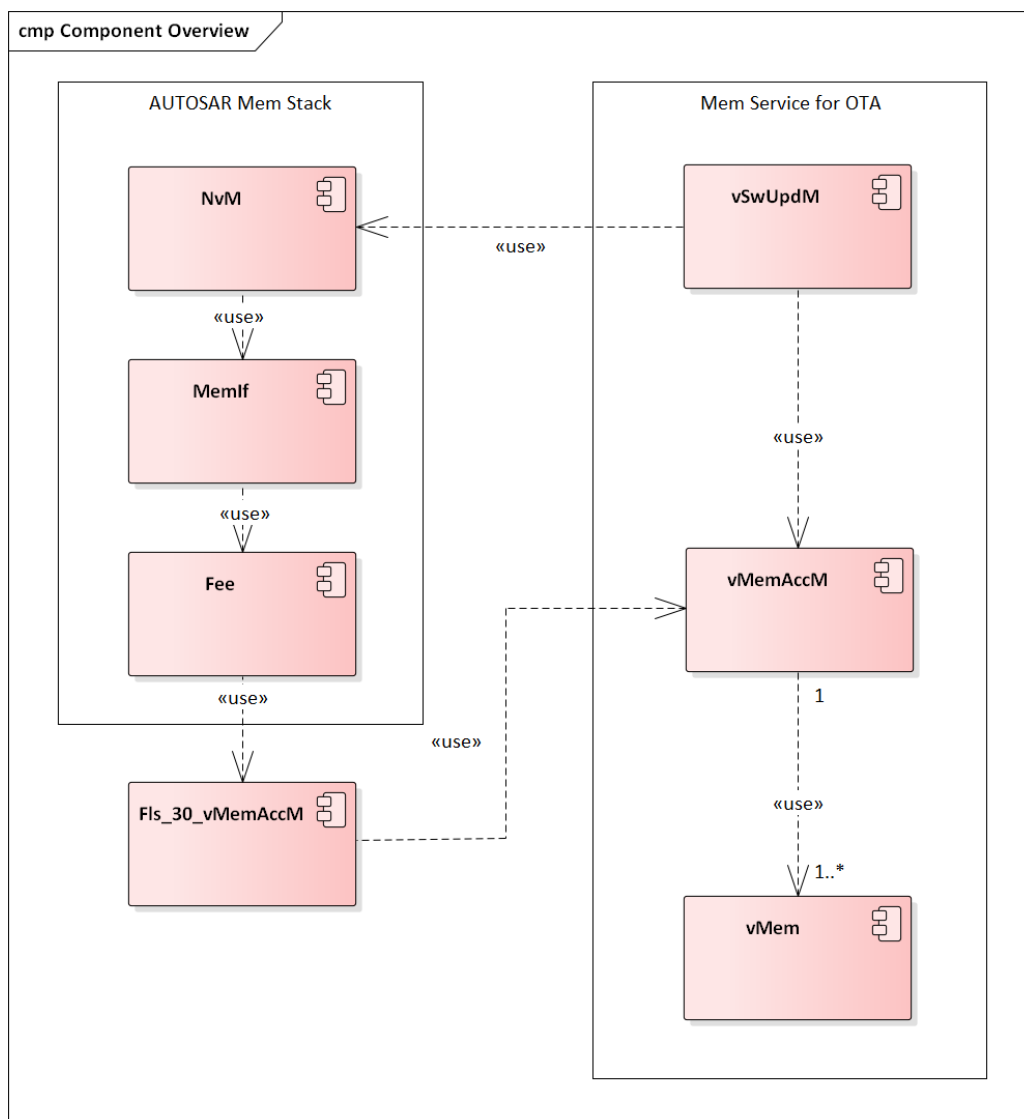


Figure 2 Component Overview of one most likely use case

1.2 Component Interfaces

The next figure shows the interfaces to adjacent modules of the vMemAccM. These interfaces are described in chapter 4.2.

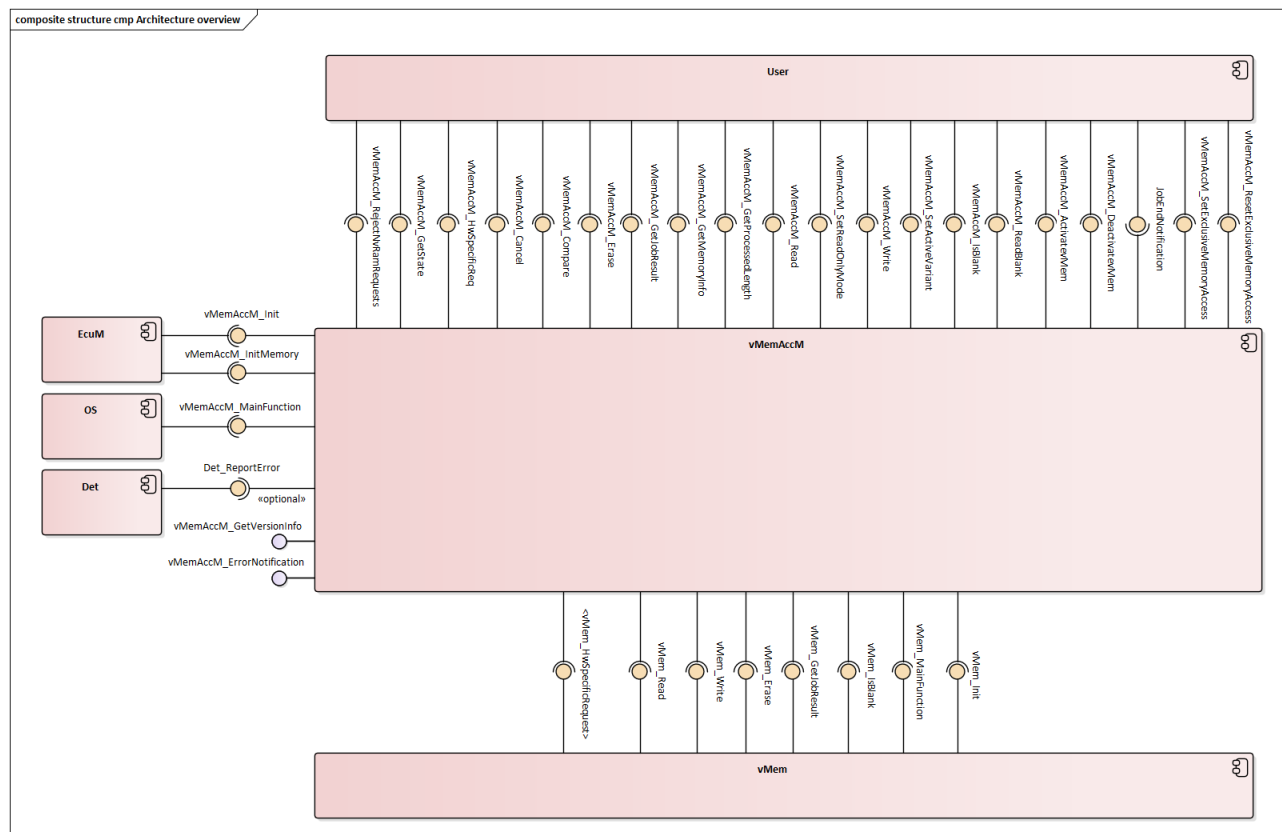


Figure 3 Interfaces to adjacent modules of the vMemAccM

2 Functional Description

2.1 Features

The features listed in the following table cover the complete functionality specified for the vMemAccM. Since the vMemAccM is not an AUTOSAR component all features provided are beyond AUTOSAR standard:

Provided Features
Read, erase, write, compare NV-data
Cancel an asynchronous job
Provide the result of the most recent asynchronous job
Pass through of hardware specific requests to underlying vMem modules
Set the read-only status of an address area
Set the active memory mapping variant. Support of address area memory mapping variants.
Queueing of jobs
Priority based job processing
Exploit burst capabilities of underlying driver
Provide information on memory device characteristics (e.g., the size of the smallest writable unit)
Provide information about processing state
Provide module version information
Check whether an address area is blank (erased and therefore writable)
Read without accessing erased NV RAM
Synchronization of multiple binaries
Control whether NV RAM can be accessed or not
Initialize underlying vMems
Schedule underlying vMems
Activate alternate vMems
Deactivate alternate vMems
Call job end notifications
Parallel job processing
Provide a minimal synchronous mode
Provide possibility to set exclusive memory access for a specific Address Area

Table 2-1 Provided Features

2.2 Initialization

The vMemAccM itself is initialized via the services `vMemAccM_InitMemory()` and `vMemAccM_Init()`. The internal variable structures only need to be initialized via the service `vMemAccM_InitMemory()`, in case they are not initialized by the startup code. During the initialization of the vMemAccM all configured vMems will also be initialized automatically.

**Caution**

If an underlying vMem targets external memory devices (e.g. via SPI) it might be necessary that during initialization information about the communication set up must be passed to this vMem via a Configuration Pointer.

If an underlying vMem actually expect a Configuration Pointer during initialization should be checked within the vMem technical references [2].

In this case the parameter

`/MICROSAR/vMemAccM/vMemAccMAddressAreaConfiguration/vMemAccMSubAddressAreaConfiguration/vMemAccMConfigPointerUsed`

shall be enabled within each SubAddressArea which references vMems expecting a Configuration Pointer.

The EcuM usually takes care of initialization. If no EcuM is used these functions have to be called by application in correct order.

2.3 Main Functions

When an asynchronous request is accepted, the job is queued and its processing requires [1, n] `vMemAccM_MainFunction()` calls. The `vMemAccM_MainFunction()` triggering needs to be done by user (mostly OS tasks).

The following are the asynchronous API services of vMemAccM:

- > `vMemAccM_Read()`
- > `vMemAccM_Write()`
- > `vMemAccM_Erase()`
- > `vMemAccM_Compare()`
- > `vMemAccM_HwSpecificReq()`
- > `vMemAccM_IsBlank()`
- > `vMemAccM_ReadBlank()`

For a more detailed description on the processing of asynchronous jobs in general, see chapters 2.7 and 2.10.

**Note**

The vMemAccM accepts at most one asynchronous job request per address area at a time.

2.3.1 Main Function triggering

`vMemAccM_MainFunction()` triggering can be configured to

- > fixed cycle time, or
- > background task.



Changes

Underlying vMems are now processed together with the `vMemAccM_MainFunction()`, so the vMems no longer need to be scheduled from the outside.

As a consequence, vMems can no longer be triggered faster than the vMemAccM.

2.3.2 Optimized Job Processing

The internal processing of the `vMemAccM_MainFunction()` is optimized in a way, that the time between two `vMemAccM_MainFunction()` shall be “used” effectively by the underlying hardware to process an asynchronous job.

Therefore, after a queued job or one of its job steps (see chapter 2.10) is finished and a following job or job step is still in the queue, it will be forwarded to the vMem/hardware within the same `vMemAccM_MainFunction()` cycle. With this behaviour, the overall needed amount of `vMemAccM_MainFunction()` and therefore the time needed to process and finish a job is minimized.



Note

In case the vMem/hardware job is finished synchronously, it is ensured by vMemAccM that only a maximum of two job (steps) are processed within one `vMemAccM_MainFunction()`.

2.4 Job Results

The vMemAccM implements different job results for informing the user about the current state of an address area and the result of the latest job. This job result can be retrieved via the service `vMemAccM_GetJobResult()`. The job result can also be obtained via the notifications, which can be configured for each address area. More information about that can be found within chapter 4.5.

2.4.1 VMEMACCM_JOB_PENDING

The job result is set to `VMEMACCM_JOB_PENDING` as soon as a request is accepted and then no further jobs will be accepted for the same address area. If the job result for an address area is different from `VMEMACCM_JOB_PENDING`, the address area is idle. That

means, the vMemAccM does not process any job for this address area and a new job can be requested.

2.4.2 VMEMACCM_JOB_OK

The job result `VMEMACCM_JOB_OK` indicates that the last job has been processed successfully. In particular, for compare jobs it means, that the data retrieved from NV memory matches the data passed by the user.

2.4.3 VMEMACCM_READ_CORRECTED_ERRORS

The job result `VMEMACCM_READ_CORRECTED_ERRORS` exists for read jobs only. This result will be returned, in case ECC errors were corrected by the device at some address within the requested read range. If any of the vMem read job steps (see 2.10) returns with result `VMEM_READ_CORRECTED_ERRORS`, the final vMemAccM job result will never become `VMEMACCM_JOB_OK`. However, the requested read data is provided and valid and therefore the read job is considered successful.

2.4.4 VMEMACCM_JOB_CANCELED

The job result `VMEMACCM_JOB_CANCELED` indicates a previously called `vMemAccM_Cancel()` service for an ongoing request. The request was aborted, the respective address area is ready for the next request.

2.4.5 VMEMACCM_JOB_FAILED

The job result `VMEMACCM_JOB_FAILED` indicates that the last job has been processed unsuccessfully.

2.4.6 VMEMACCM_READ_UNCORRECTABLE_ERRORS

The job result `VMEMACCM_READ_UNCORRECTABLE_ERRORS` exists for read jobs only. This result will be returned, in case uncorrectable ECC errors were detected by the NV memory device. Contrary to the case of corrected ECC errors, the vMemAccM will abort the read job unsuccessfully and cannot provide any valid data.

2.4.7 VMEMACCM_MEM_NOT_BLANK

The job result `VMEMACCM_MEM_NOT_BLANK` exists for is-blank jobs only. This result will be returned when at least a part of or the checked address space is not blank.

2.4.8 VMEMACCM_MEM_NOT_EQUAL

The job result `VMEMACCM_MEM_NOT_EQUAL` exists for compare jobs only. This result will be returned, if the provided data does not match the NV RAM content.

2.5 Error Notification

The related API (see chapter 4.2.18) is meant to be used to notify the vMemAccM about occurred ECC errors during a read job, mostly in case the vMem driver is not capable to do so. This is for example, if the related interrupt handles and therefore clears the ECC registers, so the vMem cannot read out the information about occurred ECC.

In such case the interrupt implementation shall check whether the address causing the interrupt belongs to the active/ ongoing vMemAccM job via `vMemAccM_GetState()` (see chapter 4.2.16) and then invoke the `vMemAccM_ErrorNotification()` with one of the allowed results:

- > `VMEMACCM_ERRORNOTIF_READ_CORRECTED_ERRORS`, if the occurred ECC errors were corrected and the data can be used,
- > `VMEMACCM_ERRORNOTIF_READ_UNCORRECTABLE_ERRORS`, if the occurred ECC leads to invalid data, vMemAccM shall abort the reading as soon as possible.

vMemAccM stores the provided result internally, waits for the vMem to finish the job and finally combines the provided and the vMem result as follows:

vMem result	Error Notification result	vMemAccM result
OK	READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS
OK	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS
READ_CORRECTED_ERRORS	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_UNCORRECTABLE_ERRORS	READ_CORRECTED_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
FAILED	READ_CORRECTED_ERRORS	FAILED
FAILED	READ_UNCORRECTABLE_ERRORS	FAILED

Table 2-2 vMemAccM and vMem result combinatorics in case of Error Notification.

Further vMemAccM behavior is described in chapter 2.4.



Caution

vMemAccM checks the preconditions before storing and using the passed result. These preconditions are

- > Address Area identifier matches the currently active one
- > Address fits to currently active address space
- > The currently active job is a read job.

If one of the preconditions does not match, vMemAccM will reject the request and will not store nor use the passed result.



Basic Knowledge

Active means the vMemAccM is currently waiting for the vMem to process its job. The vMemAccM and vMem are actively working on a specific address space and therefore all e.g. ECC interrupts caused by any address belonging to the active address space are probably caused by the vMemAccM and vMem NV RAM access.

2.6 Memory Address Mapping

One of the main responsibilities of the vMemAccM is the mapping between pseudo-physical and virtual addresses, in order to provide an abstraction of the memory resources that are actually available. Upper layers only need to use virtual addresses and don't have to be

concerned with data spreading and storage on different memory devices. Whereas the underlying memory driver can only handle the actual pseudo-physical addresses.

Examples of use cases where the memory address mapping can be particularly helpful are

- > one virtual address area shall be splitted into data and program flash, e.g. for an update of program and calibration data
- > one virtual address area shall be splitted into different non-continuous flash sectors, e.g. for an update of different program parts

The physical memory on the level of the vMem drivers is structured in **sectors**, the smallest erasable units. Identical and coherent sectors can be grouped to **sector batches**. This way a greater number of identical, contiguous sectors can be configured collectively and handled in the same way. In the vMemAccM the entire physical memory of all available sector batches is structured in virtual memory areas, the **address areas**. An address area contains **sub address areas** and each sub address area represents a continuous part of a physical sector batch. In other words, a sub address area is a part of an address area, which corresponds to a physical continuous area of memory.

For every sub address area a **virtual start address** is configured. The sub address areas within an address area do not have to be virtually continuous. That is, there can be gaps between the sub address areas within one address area. However, they must not overlap within the address area and if several sub address areas represent memory from the same sector batch, they must not overlap within the sector batch. The length of a sub address area is determined by the number of sectors it represents and their sector size. Note that different address areas may have the same virtual addresses.

Within every asynchronous job request an address area identifier, a virtual address within the respective address area and a length has to be passed to the vMemAccM. The vMemAccM maps the passed virtual address to a pseudo-physical address (vMem device instance and device specific address).



Caution

The address mapping can only be performed, if the whole virtual address range of the request is valid. Hence, asynchronous jobs can only be requested for virtual address space, which is filled with sub address areas. Requests that include gaps cannot be processed properly.

Figure 4 Memory address mapping shows an example of an address area containing four sub address areas. The virtual start address of the address area is 0x0, however its first valid address is the start address of the first sub address area. The virtual end address of the address area in this example is 0xFFFF FFFF which is also the maximum valid address. Note that the different colors of the memory areas have no meaning and are only used to underline where different sectors are mapped to. For more details on the configuration of address areas and sub address areas see chapter 5.2.

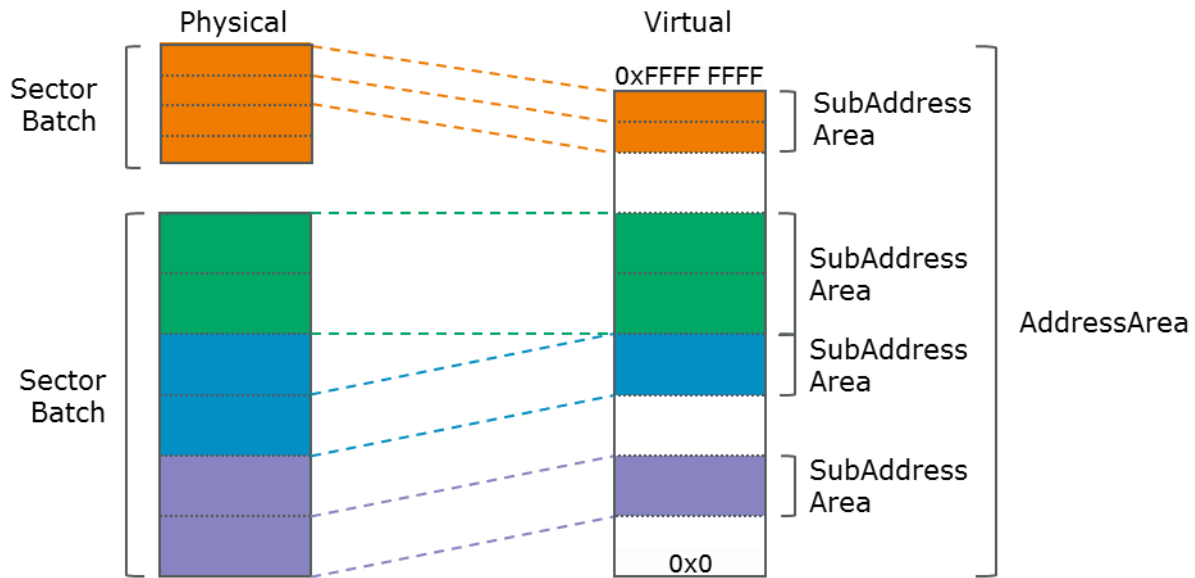


Figure 4 Memory address mapping

**Caution**

The vMemAccM checks that the whole range of an asynchronous job is valid within its internal parameter checks (see chapter 2.11).

2.6.1 Memory address mapping variants

In some use cases the virtual address needs to be remapped from one to another physical address. To achieve this the memory mapping variants were introduced.

Each address area can be configured with memory mapping variants. To set the needed memory mapping variant via appropriate API vMemAccM provides two variant identifiers:

- ▶ `VMEMACCM_VARIANT_ID_STARTUP 0u`: the startup variant, always set during the initialization (startup). Always available and valid for all Address Areas (Address Areas without variants know only one memory mapping).
- ▶ `VMEMACCM_VARIANT_ID_ALTERNATE 1u`: the alternate memory mapping. Only available, if at least one Address Area is configured with memory mapping variants.

The variant identifiers 0u and 1u are unique – 0u always refers to the startup variant, 1u to the alternate variant. The identifiers must be set within the configuration according to the needs.

**Caution**

The set variant is always valid for all address areas with variants. It is not possible to set the variant of each address area separately.



Basic Knowledge

- ▶ Each address area memory mapping variant spans the same virtual address space.
- ▶ Each address area spans all required sub address areas, each address area variant refers to the required sub address areas (configuration).
- ▶ An address area without variants must not configure the variant container, just the needed sub address area containers. Calling the `vMemAccM_SetActiveVariant` function will not affect these address areas.
- ▶ It is not possible to configure only one address area memory mapping variant, vMemAccM always needs two variants to switch between.

2.7 Parallel processing of different asynchronous jobs

The vMemAccM provides a mechanism to allow parallel processing of jobs on lower layers, so called synchronization groups. The user can configure a group of sub address areas, which must be synchronized, i.e., in this group only one job can be executed at a time.

This mechanism can be used to allow parallel processing of a job operating on the internal flash and simultaneously executing a job on external flash. Otherwise, an access to an external flash blocks all internal jobs as well.

For configuration hints see chapter 5.2.3.



Note

This feature can only be used for different address areas. Multiple synchronization groups that are part of a single address area will **not be** parallelized.



Note

This feature does not work in multiple binary use cases.

2.8 Priority Based Job Processing

The vMemAccM accepts at most one asynchronous job request per address area at a time. Since the vMemAccM will process only one asynchronous job per synchronization group and per address area at a time, it implements a queue of size number of address areas. An asynchronous job request is only accepted and queued if there is no other currently pending

job for the requested address area. So, if for instance a read job for an address area is queued, an erase job for the same address area will not be accepted before the read job is finished.

Each address area has its own priority which can be configured. Note that the priorities of the address areas must be pairwise different. The priorities 0 (lowest priority) until 65535 (highest priority) can be selected for an address area.

An asynchronous job on an address area with higher priority is processed before asynchronous jobs on address areas with lower priority. If there is an already ongoing asynchronous job on an address area with lower priority, it will be suspended as soon as possible. Note that an already ongoing asynchronous job on the underlying vMem will not be interrupted.

2.9 Burst Feature

The underlying vMem driver may offer burst capabilities, that is instead of writing/erasing one smallest possible unit a number of these units are written/erased at once to increase performance.

The smallest writable physical memory unit is usually called a **page**. To improve performance, the underlying vMem modules may offer a **write burst** feature depending on the hardware capabilities. Instead of writing one page at a time, multiple pages within the same sector batch can be written at once.

The smallest erasable physical memory unit is usually called a **sector**. To improve performance, the underlying vMem modules may offer an **erase burst** feature, similarly as for write jobs. Instead of erasing one sector at a time, multiple sectors within the same sector batch can be erased at once.

If the underlying vMem module is burst capable, the user of the vMemAccM can chose during configuration whether the write or erase burst feature shall be exploited by the vMemAccM. The advantage of using the burst feature is certainly the increased performance. However, the user may still not want the vMemAccM to burst, because of too long burst durations. For example, it might happen that a job with low priority blocks an address area for a job with high priority for a longer time, because burst is used.

2.10 Partitioning of Asynchronous Jobs

Note that in this document the **length of an asynchronous job** always refers to the number of bytes of data which shall be processed. This information is always passed as a parameter of an asynchronous request.

To avoid blocking requests the vMemAccM splits asynchronous job requests into smaller jobs which are forwarded to the underlying vMem modules. An asynchronous job for an address area will be divided into at least the number of involved sub address areas, since each sub address area might correspond to an entirely different sector batch. But even requests which involve only one sub address area are divided into job steps of shorter length. The step length depends on the request and configuration parameters. In the following the partitioning of the individual asynchronous jobs are described in more detail.

2.10.1 Read

The vMemAccM divides the read request into smaller jobs which are forwarded to the underlying vMem modules. Theoretically the vMem may accept read requests with any length, but this way the related vMem is only blocked for a shorter time. The maximum number of bytes the vMemAccM module requests within one vMem read request can be configured per address area (see chapter 5.2.1).

2.10.2 ReadBlank

ReadBlank jobs are partitioned into smaller jobs. Every page of the given address range needs to be blank checked and will only be read if it is valid to do so. Since blank checks have to be performed page wise, the configured number of bytes (see chapter 6.2.1) for a vMem read request cannot be taken into account.



Caution

There is no burst for the blank checks. Therefore, every single page has to be checked one after another. This makes reading long address ranges very time consuming. Try to only read short memory areas like important data patterns.

2.10.3 Write

Write jobs are partitioned into smaller jobs of length write burst size or page size.

2.10.4 Erase

Erase jobs are partitioned into smaller jobs of length erase burst size or sector size.

2.10.5 Compare

The compare job differs from the other asynchronous jobs, since the underlying vMem modules do not offer a compare service. Hence the vMemAccM cannot simply forward the compare request to the vMem but has to read the data in question using the vMem read service and store it in an internal buffer (see chapter 2.12). The vMemAccM then compares the data read to the data passed by the user. Therefore, the maximum number of bytes the vMemAccM can compare at a time is determined by the maximum read request length (of the related address area).

2.10.6 IsBlank

IsBlank jobs are partitioned into smaller jobs of page size.

2.10.7 Hardware Specific Requests

It is possible that the underlying vMem modules offers other special features to access memory besides the basic ones (read, write and erase), depending on the individual hardware. Since any kind of memory access shall pass through the vMemAccM, the vMemAccM offers a special asynchronous service `vMemAccM_HwSpecificReq()`, which allows upper layers to use these special features.

The hardware specific requests are treated by the vMemAccM as any other asynchronous job, that is no other asynchronous jobs will be accepted for the corresponding address area until the job is completed.

Information on a specific vMem hardware instance, such as the unique hardware identifier, can be obtained via the service `vMemAccM_GetMemoryInfo()`. The unique hardware identifier is needed as a parameter in the service `vMemAccM_HwSpecificReq()`.

**Caution**

The vMemAccM interface shall be open for any vMem implementation and cannot be responsible for partitioning the hardware specific requests into jobs of smaller length. Therefore, vMemAccM will neither check nor access the passed data pointer and its size. The values will be forwarded directly to the vMem.

**Note**

If the vMem request is basically a void-void function, it still must match the signature. In this case the pointer can be `NULL_PTR` and the size of the pointer can be 0.

2.11 Retries of a failed job

To resolve transient errors vMemAccM retries a failed vMem request.

Since a vMemAccM request normally leads to several vMem requests, the retries will always be done on the one failed vMem request setting up the same request (instance identifier, physical address, buffer address and length) again and again until the vMem finishes the job successfully, or the retry count exceeds the configured maximum value.

The user will not be notified about the retrying. Retrying the job vMemAccM ensures a failed request was not caused by transient errors, but caused by a potentially real hardware problem. The user shall behave according to the available failure handlings (depends on used hardware), e.g. for a failed write the sector shall be erased and then the data written again.

**Practical Procedure**

Find the address causing the failure:

The failing address can be calculated via the successfully processed request length (in bytes) published via `vMemAccM_GetProcessedLength`. The length always shows the successfully processed number of bytes.

If the job result matches an unsuccessful job and the returned length is lower than the requested length, the length + 1 marks the failing address offset.

**Basic Knowledge**

vMemAccM does not retry rejected jobs.

vMem may reject a burst size request, where vMemAccM retries the job setup with normal size, assuming vMem cannot burst right now because e.g. not aligned RAM buffer.

But after the vMem rejects a normal size request, vMemAccM assumes the vMem is not operable and aborts processing unsuccessfully.

2.11.1 Retry a failed burst size request

A burst size vMem request consists of several pages (write) or sectors (erase). To find the most precise address of the failure, vMemAccM uses a slightly different retry strategy than for normal size requests: split the burst size into normal size jobs and process all these normal size jobs before using the burst feature again.

If there is a real hardware failure, one of the normal size requests will fail and the `vMemAccM_GetProcessedLength` function will deliver the successfully processed length as described in previous chapter.

If all the normal size requests succeed, vMemAccM will assume the failure was caused by a transient error and continue processing with the next step (normal size or burst size – depending on current job progress).

2.11.2 Retry configuration

Because the possible number of retries depends on hardware, the value must be configured within a sub address area as the mapping element from virtual to physical address space and therefore to the hardware.

The configured number of retries is always valid for one physical address, that means once a vMem request to a specific address succeeds, the next vMem request to the next address will use its own retry counter.

Number of retries 0 disables retries, vMemAccM will never retry a failed vMem request. 1 forces vMemAccM to retry a failed vMem request once, and so on.

2.11.3 Supported vMemAccM requests

- ▶ vMemAccM retries a failed write job step.

- ▶ vMemAccM retries a failed erase job step.

**Caution**

To use the retries, the hardware must support

- > overwriting the physical content with the same data (part of the data may already be written correctly!)
- > erasing an already erased physical area.

2.12 RAM Buffers

vMemAccM never copies the data to another RAM buffer before forwarding the buffer to vMem – it uses the provided buffer directly to read and write data.

Only for the compare service, the vMemAccM uses an internal buffer. vMemAccM will then forward the internal buffer to the vMem to get the data from NV RAM and compare the read data against the provided user data.

The internal buffer length matches the largest `vMemAccMReadRequestLength` (see chapter 5.2.1). If the function `vMemAccM_Compare()` is disabled via the parameter `vMemAccMCompareApi`, the internal buffer is neither reserved nor used at all.

**Caution**

vMemAccM does not ensure device related RAM alignments, upper layers must take care about the correctly aligned RAM start addresses.

vMemAccM uses the internal buffer only to read data from NV RAM, the buffer will never be used to write data. Therefore, the internal buffer does not fulfill any specific RAM alignments.

2.13 Multi binary synchronization – NV RAM synchronization concept

**Changes**

The naming within this chapter was updated from “Multi partition” to “Multi binary” to reflect more precisely the synchronization capabilities of the vMemAccM.

The vMemAccM can be configured to additionally synchronize access to the NV RAM between different binaries but not between different partitions.

Note that the naming of configuration parameters as well as memory mapping sections will not change due to compatibility.

vMemAccM implements a multi binary synchronization, where multiple separate binaries access the NV RAM on their own and the vMemAccM takes care, that only one binary does so at a time.

One of the contributing binaries must be the master, all others are slaves. The master synchronizes the accesses and sets the access right to one of the binaries requesting a NV RAM access.

The current implementation does not use any operating system functionalities.

Figure 5 shows an example of the multi binary synchronization within the memory stack architecture.

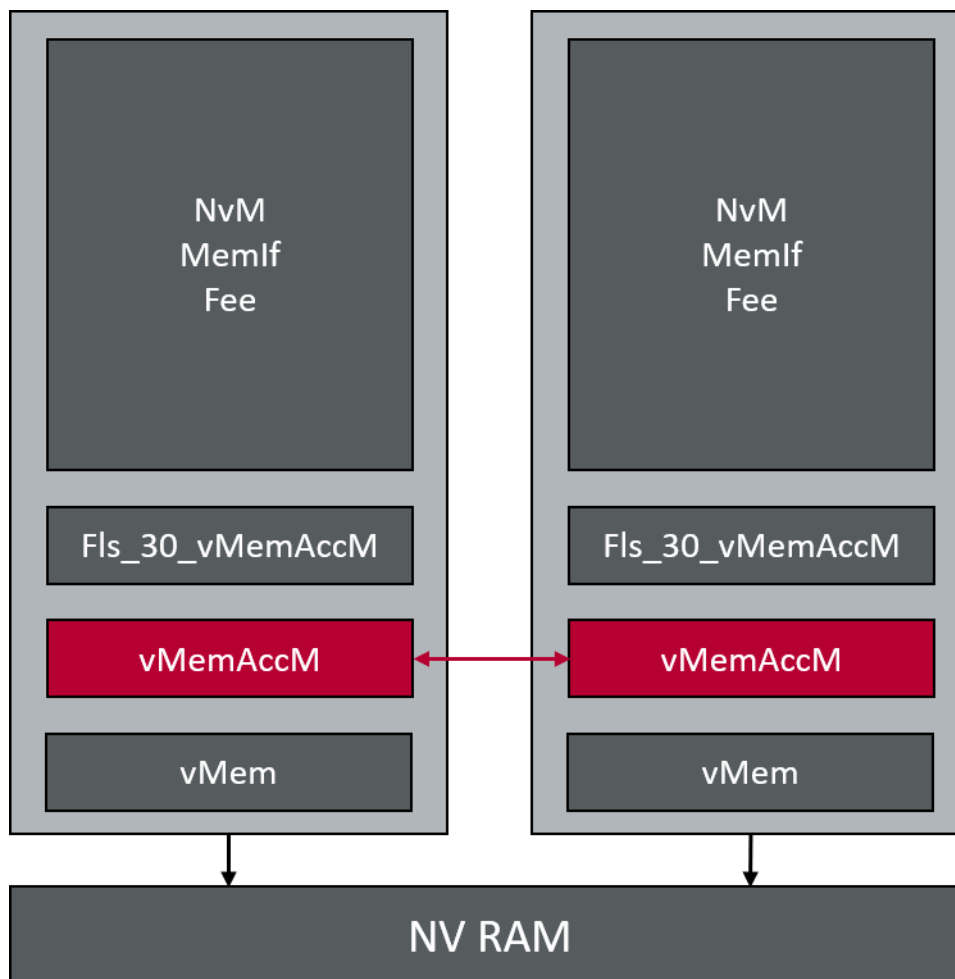


Figure 5 Multi binary synchronization architecture example (2 binaries)

**Example**

If one binary must keep its data locally, so the other binary cannot read the data, two memory stacks may be used in parallel.

If the Flash device does not support parallel access from two separated users, the access needs to be synchronized, which is possible via the vMemAccM.

Example of such a situation: HSM binary and application binary.

If both binaries can access the NV RAM in parallel, this feature is not needed!

The following chapters describe how to configure the vMemAccM module and the required variables.

2.13.1 Configuration

A vMemAccM will be the multi binary synchronization master, if the vMemAccMMasterPartitionConfiguration container exist. Otherwise, the vMemAccM will be a multi binary synchronization slave.

2.13.1.1 Master

A master, the one taking care of NV RAM access synchronization, needs to know all slaves contributing in the NV RAM access synchronization.

If the configuration contains the vMemAccMMasterPartitionConfiguration, the vMemAccM is the master and all slaves contributing in the NV RAM access synchronization need to be configured as vMemAccMPartitionConfiguration.

As master the vMemAccM automatically generates the variables needed for the synchronization:

- > vMemAccM_NVRAMAccessToken holding the current binary identifier with the NV RAM access, or synchronization information.
- > One vMemAccM_PublishReq_<vMemAccMPartitionConfiguration><vMemAccMPartitionId> per binary. The slave publishes its state to the master via this variable: slave wants to access the NV RAM, or is idle (no jobs requested).

These variables need to be mapped into appropriate shared RAM, see chapter 2.13.2 for more information.

Please see the next chapter for information about slave configuration.

2.13.1.2 Slave

A slave, the one requesting NV RAM access and waiting for the token to hold the own binary identifier, does not declare any variables. The user needs to configure shared RAM addresses or linker symbols pointing to the variables created by the master.

These are:

- > vMemAccMNVRAMAccessToken pointing to the vMemAccM_NVRAMAccessToken variable.

- > vMemAccMPartitionPublishRequest pointing to the appropriate vMemAccM_PublishReq_<vMemAccMPartitionConfiguration><vMemAccMPartitionId> variable.

A slave vMemAccM does not contain the vMemAccMMasterPartitionConfiguration container. Since the master configuration always requires all slaves, a slave configuration shall always match one of the slaves available in master configuration!

The vMemAccMNVRAMAccessToken, vMemAccMPartitionPublishRequest and also the vMemAccMPartitionId must match the master configuration.

2.13.2 Memory Mapping

The multi binary synchronization variables declared by the vMemAccM master binary must be mapped into appropriate shared RAM region(s). The vMemAccM memory mapping sections always follow the naming schema:

- > VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
- > VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>.

The sections always hold the following variables:

- > VMEMACCM_START/STOP_SEC_MULTIPARTITION_PARTITION_ID0
 - The NV RAM access token where the master binary writes, all other slave binaries read.
- > VMEMACCM_START/STOP_SEC_MULTIPARTITION_PARTITION_ID<1...n>
 - The request value published/ written by the respective slave binary, the master binary reads the value.

If the memory mapping sections are unmapped, for example after configuring the vMemAccM synchronization for the very first time, the vMemAccM memory sections will cause a compiler error. This will be done to notify the integrator about the missing memory mapping pragmas.

2.13.2.1 Examples

Following sections describe how the mapping can be done for specific compilers. These are just examples, please refer to the appropriate compiler documentation for more information.

**Example:** Windriver DiabData Compiler

```
#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section DATA ".vMemAccMNVRAMAccessToken" ".vMemAccMNVRAMAccessToken"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section DATA
# define STOP_SEC_VAR
#endif

#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section DATA ".vMemAccMPublishRequest<PartitionId>"
".vMemAccMPublishRequest<PartitionId>"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section DATA
# define STOP_SEC_VAR
#endif
```

**Example:** Green Hills Compiler

```
#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma ghs section data=".vMemAccMNVRAMAccessToken"
# pragma ghs section sdata=".svMemAccMNVRAMAccessToken"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma ghs section data=default
# pragma ghs section sdata=default
# define STOP_SEC_VAR
#endif

#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma ghs section data=".vMemAccMPublishRequest<PartitionId>"
# pragma ghs section sdata=".svMemAccMPublishRequest<PartitionId>"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma ghs section data=default
# pragma ghs section sdata=default
# define STOP_SEC_VAR
#endif
```

**Example:** Altium TASKING Compiler

```
#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section all "vMemAccMNVRAMAccessToken"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section all restore
# define STOP_SEC_VAR
#endif

#ifndef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section all ".vMemAccMPublishRequest<PartitionId>"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifndef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section all restore
# define STOP_SEC_VAR
#endif
```

2.14 Multi binary synchronization – Satellite concept

**Changes**

The naming within this chapter was updated from “Multi partition” to “Multi binary” to reflect more precisely the synchronization capabilities of the vMemAccM.

The vMemAccM can be configured to additionally forward jobs from a satellite binary to a master binary but not between different partitions.

Note that the naming of configuration parameters as well as memory mapping sections will not change due to compatibility.

vMemAccM implements a synchronization feature, which enables binaries to access the underlying NV RAM, even if they do not directly (physically) have access to it.

The constellation needs a master, which has access to the NV RAM. The satellites will not access the NV RAM directly, but will instead write the data through the master.

Figure 6 shows an example of the multi binary synchronization within the memory stack architecture.

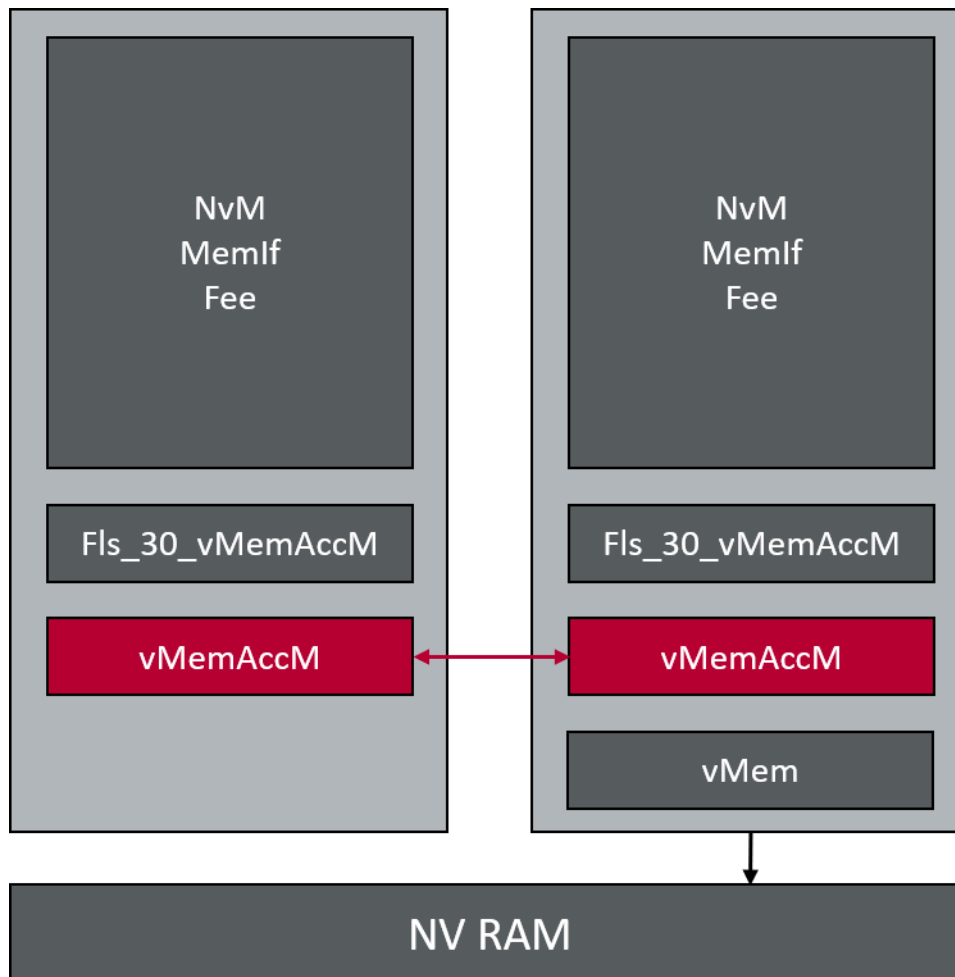


Figure 6 Multi binary satellite architecture example (2 binaries)



Example

If one binary does not have access to NV RAM but needs to persist data, it can be configured as a satellite to access the NV RAM through the master vMemAccM.

Example of such a situation: System on a chip without direct access to NV RAM.

If both binaries can access the NV RAM, this feature is not needed! Consider Chapter 2.13 instead.

**Caution**

The data will be transmitted unchanged to the master. If the data contains sensitive information, it will need to be encrypted before writing to the NV RAM.

**Caution**

The vMemAccM satellites do not support hardware specific requests.

**Note**

The NV RAM access will generally be slower due to synchronization overhead and potentially higher priority of other requested jobs. The priority of the jobs depend on the configured address area of a memory section. The satellites currently do not support bursting, which means a further decrease in performance in certain scenarios.

2.14.1 Configuration

The configuration is the same as described in chapter 2.13.1 for the most part. The following differences apply:

The value of

`/vMemAccMMultiPartitionConfiguration/vMemAccMMultiPartitionUseCase` must be set to `SynchronizeSingleNVRAMAccessor` instead of the default value.

Since there are (at least) two configurations (one for the master, one for each satellite), certain values will have to be duplicated. Address areas configured within the satellites need to be configured inside the master as well. The master has to know about all the available address areas and their values. Additionally, all address areas need to be unique and should not overlap. Take care when configuring them. The satellites only need to know their own address areas, but it makes sense to synchronize all configurations (all the master and satellite configurations) and have everything together in each configuration. This way, the configurations all need to be consistent but will make less problems due to missing address area identifiers for example.

2.14.1.1 Configuration tips

2.14.1.1.1 Configuration with vMem

Since all configurations need to have address areas configured, it is necessary that each vMemAccM has a vMem in the configuration as well. This is because the vMemAccM has references to the vMem, which it uses for the configuration. Since only the master shall be

able to access the NV RAM, the satellites do not need a working vMem, since no code needs to be generated. It is solely necessary for the configuration part. Any vMem can be used to achieve this.

2.14.1.1.2 Configuration with Fls_30_vMemAccM

The suggested way to configure the Fls_30_vMemAccM is depicted within Figure 7. Each Fls_30_vMemAccM should use a different address area in the configuration to avoid confusion and problems.

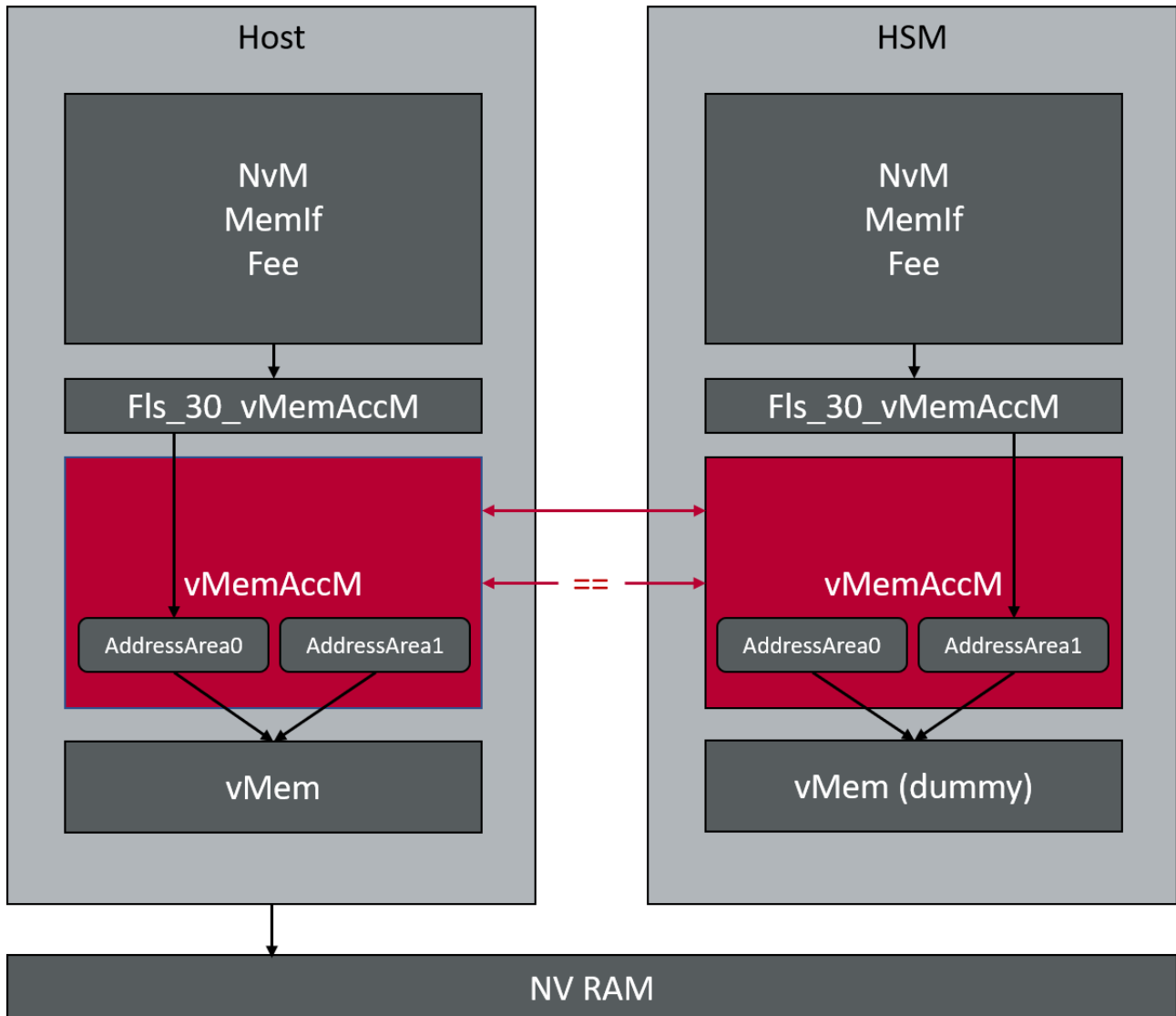


Figure 7 Configuration of Fls_30_vMemAccM

2.14.1.2 Other tips

When using a controller with a System-on-a-chip a Region-based Address Translation (**RAT**) might be used. Please refer to the data sheet of the used controller to find out if the project might be affected by this.

Generally, the vMemAccM configuration should not be affected by this, since symbol names can be configured. If a static address is used instead of a symbol, it might be different, so take care when configuring the values.

2.15 Activate alternate vMems

Sometimes it is desirable to have limited functionality with vMems. It might be enough to only read data most of the time and disable the write functionality completely. This ensures that no data can accidentally be persisted. However, even in this use case it might be necessary to sporadically persist data. This can be covered by having a configured vMem which can only read and an alternate vMem with full access which will be downloaded. The vMem with full access can then be activated through the `vMemAccM_ActivatevMem` API to persist data when necessary and then be deactivated with the `vMemAccM_DeactivatevMem` API again once the read-only mode is enough.

Such alternate vMems can be stored in a RAM buffer, or somewhere else in NV RAM. The idea is to temporarily replace the configured vMem with such an alternate vMem, use the alternate vMem and then go back to the configured vMem.

A further possible use case for this functionality could be to provide a newer or fixed vMem or one with extended functionality.



Note

This functionality needs to be explicitly enabled in the configuration via the `/MICROSAR/vMemAccM/vMemAccMGeneral/vMemAccMUseDownloadableVMem` parameter.

If the vMem which shall be activated/deactivated targets external memory devices (e.g. via SPI) the parameter `/MICROSAR/vMemAccM/vMemAccMAddressAreaConfiguration/vMemAccMSubAddressAreaConfiguration/vMemAccMConfigPointerUsed` shall be additionally enabled within each SubAddressArea which references this vMem. For more information see the technical reference of the underlying vMem drivers [2].

This option is not available in the multi binary single accessor satellite use case. See chapter 2.14 for more information about that use case.

2.15.1 vMem validation

Before activating a vMem, the data in the memory will be validated first.

Each vMem starts with a header, which includes information to identify it. For more information, please refer to the documentation of the vMem.

A footer is placed at the end of each vMem, which contains the same information as the header.

Both of these structures are used to enclose the vMem in memory.

Should the data not be correct, the validation fails and the vMem will not be activated. One reason for such a failure could be an invalid version.

2.16 Error Handling

2.16.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1], if development error reporting is enabled (i.e. pre-compile parameter `VMEMACCM_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but must have the same signature as the service `Det_ReportError()`.

The reported vMemAccM ID is 255.

The reported service IDs identify the services which are described in detail in 0. The following table presents the service IDs and the related services:

Service ID	Service
0x00	<code>vMemAccM_InitMemory()</code>
0x01	<code>vMemAccM_Init()</code>
0x02	<code>vMemAccM_Read()</code>
0x03	<code>vMemAccM_Write()</code>
0x04	<code>vMemAccM_Erase()</code>
0x05	<code>vMemAccM_Compare()</code>
0x06	<code>vMemAccM_HwSpecificReq()</code>
0x07	<code>vMemAccM_MainFunction()</code>
0x08	<code>vMemAccM_Cancel()</code>
0x09	<code>vMemAccM_GetJobResult()</code>
0x0A	<code>vMemAccM_GetMemoryInfo()</code>
0x0B	<code>vMemAccM_SetReadOnlyMode()</code>
0x0C	<code>vMemAccM_GetProcessedLength()</code>
0x0D	<code>vMemAccM_GetVersionInfo()</code>
0x0E	<code>vMemAccM_GetState()</code>
0x0F	<code>vMemAccM_SetActiveVariant()</code>
0x10	<code>vMemAccM_ErrorNotification()</code>
0x11	<code>vMemAccM_IsBlank()</code>
0x12	<code>vMemAccM_ReadBlank()</code>
0x13	<code>vMemAccM_RejectNvRamRequests()</code>
0x14	<code>vMemAccM_ActivatevMem()</code>
0x15	<code>vMemAccM_DeactivatevMem()</code>
0x16	<code>vMemAccM_SetExclusiveMemoryAccess()</code>

Service ID	Service
0x17	vMemAccM_ResetExclusiveMemoryAccess()

Table 2-3 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x01 VMEMACCM_E_UNINIT	An API service has been called before component was initialized.
0x02 VMEMACCM_E_PARAM_POINTER	An API service has been called with an NULL pointer.
0x03 VMEMACCM_E_PARAM_ADDRESS_AREA_ID	An API service has been called with an invalid address area ID.
0x04 VMEMACCM_E_PARAM_ADDRESS_LENGTH	An API service has been called with an invalid virtual address and / or length. For instance, the virtual address is not within the related address area or the requested length exceeds the length of the address area.
0x05 VMEMACCM_E_PENDING	An asynchronous API service was called, while another job of requested address area is already being processed. Only one asynchronous job request is accepted per address area.
0x06 VMEMACCM_E_PARAM_HW_SPECIFIC_FUNC_INDEX	The passed HW specific function index does not belong to passed address area ID.
0x07 VMEMACCM_E_PARAM_HW_ID	The passed HW identifier does not belong to passed address area ID.
0x08 VMEMACCM_E_PARAM_VARIANT_ID	The passed memory mapping variant identifier does not exist.
0x09 VMEMACCM_E_ADDRESS_AREA_NOT_ACTIVE	The passed address area ID is not active in processing sub module.
0x0A VMEMACCM_E_EXCLUSIVE_MEMORY_ACCESS_ALREADY_SET	Exclusive memory access has already been set for another address area.

Table 2-4 Errors reported to DET

2.16.1.1 Parameter Checking

The following table shows which parameter checks are performed on which services:

Service	Check/Error code	VMEMACCM_E_UNINIT	VMEMACCM_E_PARAM_POINTER	VMEMACCM_E_PARAM_ADDRESS_AREA_ID	VMEMACCM_E_PARAM_ADDRESS_LENGTH	VMEMACCM_E_PENDING	VMEMACCM_E_PARAM_HW_SPECIFIC_FUNC_INDEX	VMEMACCM_E_PARAM_HW_ID	VMEMACCM_E_PARAM_VARIANT_ID	VMEMACCM_E_ADDRESS_AREA_NOT_ACTIVE	VMEMACCM_E_EXCLUSIVE_MEMORY_ACCESS_ALREADY_SET
vMemAccM_InitMemory()											
vMemAccM_Init()											
vMemAccM_Read()		■	■	■	■	■					
vMemAccM_Write()		■	■	■	■	■					
vMemAccM_Erase()		■		■	■	■					
vMemAccM_Compare()		■	■	■	■	■					
vMemAccM_HwSpecificReq()		■	■	■		■	■	■			
vMemAccM_MainFunction()		■									
vMemAccM_Cancel()		■		■							
vMemAccM_GetJobResult()		■		■							
vMemAccM_GetMemoryInfo()		■	■	■							
vMemAccM_SetReadOnlyMode()		■		■							
vMemAccM_GetProcessedLength()		■	■	■							
vMemAccM_GetVersionInfo()			■								
vMemAccM_GetState()		■	■	■							
vMemAccM_IsBlank()		■		■	■	■					
vMemAccM_SetActiveVariant()		■				■			■		
vMemAccM_ErrorNotification()		■		■						■	
vMemAccM_ReadBlank()		■	■	■	■	■					
vMemAccM_RejectNvRamRequests()		■									
vMemAccM_ActivatevMem()		■		■	■						

Service	Check/Error code									
	VMEMACCM_E_UNINIT									
	VMEMACCM_E_PARAM_POINTER									
	VMEMACCM_E_PARAM_ADDRESS_AREA_ID									
	VMEMACCM_E_PARAM_ADDRESS_LENGTH									
	VMEMACCM_E_PENDING									
	VMEMACCM_E_PARAM_HW_SPECIFIC_FUNC_INDEX									
	VMEMACCM_E_PARAM_HW_ID									
	VMEMACCM_E_PARAM_VARIANT_ID									
	VMEMACCM_E_ADDRESS_AREA_NOT_ACTIVE									
	VMEMACCM_E_EXCLUSIVE_MEMORY_ACCESS_ALREADY_SET									
vMemAccM_DeactivatevMem()	■		■	■						
vMemAccM_SetExclusiveMemoryAccess()	■		■							■

Table 2-5 Development Error Reporting: Assignment of checks to services

2.17 Minimal synchronous mode

The vMemAccM provides a minimal synchronous mode in which only the most basic functionality (see list of available services below) is available. The advantage of this mode is that the vMemAccM requires as little ROM size as possible. The most likely use case for this is the memory access by the HSM. Here, some functionality e.g., ErrorNotification, GetProcessedLength, asynchronous hardware specific request and ReadBlank service, is not always required. Instead on certain platforms with little available ROM size this additional functionality makes the vMemAccM unusable.

In the minimal synchronous mode only the services

- vMemAccM_InitMemory(),
- vMemAccM_Init(),
- vMemAccM_Read(),
- vMemAccM_Write(),
- vMemAccM_Erase(),
- vMemAccM_IsBlank() and
- vMemAccM_GetJobResult()

are available. They provide the functionality described in chapter 4.2 and have the same parameters. In difference to the description in chapter 4.2 the services vMemAccM_Read(),

`vMemAccM_Write()`, `vMemAccM_Erase()` and `vMemAccM_IsBlank()` are synchronous. So, the job is finished when the program returns from the function call.

The function `vMemAccM_MainFunction()` is available for compatibility, but calling the function is not necessary, as jobs are synchronously processed. The function does nothing. The same is true for the function `vMemAccM_Cancel()`; it is available for compatibility reasons only, the function itself does nothing. Since jobs are processed synchronously, it is impossible to call `vMemAccM_Cancel()` while a job is being processed.

**Note**

The minimal synchronous mode needs to be enabled explicitly via the configuration parameter `/MICROSAR/vMemAccM/vMemAccMGeneral/vMemAccMMinimalSync`.

All general features, except a very basic implementation of the services listed above, are not supported in the minimal synchronous mode (e.g., no retry mechanism, no job end notifications, ...).

**Caution**

It must be ensured by the user of the vMemAccM that at any time only one synchronous service is active.

Especially in the case, when more than one AddressArea is configured: If a synchronous request is still active (therefore has not yet returned), it is not allowed to call a second request for another AddressArea.

**Caution**

The minimal synchronous mode supports only "direct hardware" vMems. In this mode the jobs are synchronously processed, therefore a communication via e.g. `vMem_30_XXspi` or an adapter vMem (e.g. `vMem_30_Fls`) cannot be used as they require additional MainFunction triggering.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR vMemAccM into an application environment of an ECU.

3.1 Scope of Delivery

The delivery of the vMemAccM contains the files which are described in the chapters 3.1.1 and 3.1.2:

3.1.1 Static Files

File Name	Description
vMemAccM.c	This is the source file of the vMemAccM. It contains the implementation of the vMemAccM interfaces.
vMemAccM.h	This is the header file of the vMemAccM. It declares the interfaces of vMemAccM.
vMemAccM_Types.h	This file defines general types used by vMemAccM.
vMemAccM_vMemApi.h	This file defines the interfaces of all functions which must be provided by the vMem modules so that vMemAccM can use them. All vMem instances must implement these functions.
vMemAccM_MemAb.h vMemAccM_MemAb.c vMemAccM_CfgAb.h vMemAccM_CfgAb.c vMemAccM_StateMachines.h vMemAccM_StateMachines.c vMemAccM_ErrorNotification.h vMemAccM_Sched.h vMemAccM_Sched.c vMemAccM_SMTTypes.h vMemAccM_AltvMem.h vMemAccM_AltvMem.c vMemAccM_SchedSync.h vMemAccM_SchedSync.c vMemAccM_SMLib.h vMemAccM_SMLib.c	These are files for internal use of the vMemAccM.

Table 3-1 Static files

3.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5.

File Name	Description
vMemAccM_Cfg.h	This file contains the public configuration parameters of the vMemAccM.
vMemAccM_Lcfg.c	This file contains configuration parameters.
vMemAccM_vMemCfg.h	This file contains the vMemAccM_HwIdType.

File Name	Description
vMemAccM_IntCfg.h	This file contains internal configuration parameters.

Table 3-2 Generated files

3.1.3 Interfaces of the vMem

As described in chapter 3.1.1 the vMemAccM contains a file vMemAccM_vMemApi.h specifying the interfaces of the vMem. In addition the vMemAccM contains the description file vMem_Base_bswmd.arxml which must be refined by every vMem module.

3.2 Critical Sections

To protect critical code against interruptions vMemAccM uses the following exclusive area:

> `VMEMACCM_EXCLUSIVE_AREA_0`

The exclusive area covers only some statements and shall have a short runtime. The critical section is used vMemAccM internally and does not cover calls to other modules. vMemAccM does not nest critical sections.

3.3 Handling of non-correctable ECC errors



Caution

Depending on the platform it might be necessary for the integration to implement ECC interrupts/exceptions to handle non-correctable ECC errors.
E.g., when using vMem_30_Trave2Cyp01 for Traveo II.

For more information see 2.5 as well as the technical references of the underlying vMem drivers [2].

4 API Description

For an interfaces overview please see Figure 3.

4.1 Type Definitions

The types defined by the vMemAccM are described in this chapter.

Type Name	C-Type	Description	Value Range
vMemAccM_AddressAreaIdType	uint16	Type for the unique numeric ID of an AddressArea. The IDs are generated as defines using the symbolic name value.	0 – 65535
vMemAccM_AddressType	uint32	Type for the virtual address within an AddressArea.	0 – 4294967295
vMemAccM_LengthType	uint32	Type for the length of an asynchronous job.	0 – 4294967295
vMemAccM_vMemHwSpecificFuncIndexType	uint32	Type for index of HW specific function within HW specific function table.	0 – 4294967295
vMemAccM_LengthPtrType	uint32*	Type used to return the processed length of an asynchronous job.	Must not be NULL pointer.
vMemAccM_DataPtrType	void*	Type for pointer to buffer passed by user in read request where vMemAccM shall write the requested memory content to.	Must not be NULL pointer.
vMemAccM_ConstDataPtrType	void*	Type for pointer to buffer passed by user in write request.	Must not be NULL pointer.
vMemAccM_JobResultType	enum	Type for the asynchronous job results. The result can be polled via vMemAccM_GetJobResult() for every AddressArea and can have the following values.	VMEMACC_JOB_OK The last asynchronous job has been finished successfully. This is the default value after initialization. VMEMACC_JOB_PENDING An asynchronous job is currently being processed or within queue. VMEMACC_JOB_CANCELED The last asynchronous job has been canceled by vMemAccM_Cancel(). VMEMACC_JOB_FAILED

Type Name	C-Type	Description	Value Range
vMemAccM_ErrorNotificationResultType	enum	Type providing all possible values for vMemAccM_ErrorNotification().	The last asynchronous job has been finished unsuccessfully.
			VMEMACC_READ_CORRECTED_ERRORS ECC errors were detected and corrected during the last read job.
			VMEMACC_READ_UNCORRECTABLE_ERRORS Uncorrectable ECC errors were detected during the last read job.
			VMEMACCM_MEM_NOT_BLANK The passed address area is not blank.
			VMEMACCM_MEM_NOT_EQUAL The passed data do not match the NV RAM content.
			VMEMACCM_ERRORNOTIF_READ_CORRECTED_ERRORS ECC errors were detected and corrected during the last read job.
			VMEMACCM_ERRORNOTIF_READ_UNCORRECTABLE_ERRORS Uncorrectable ECC errors were detected during the last read job.
vMemAccM_HwIdType	enum	Type for the unique numeric identifiers of all vMem hardware instances used for hardware specific requests.	0 – 4294967295 The name of each enum parameter is constructed from the vMem module name and the vMem instance name.
vMemAccM_JobType	enum	Type for the asynchronous jobs – lists all available asynchronous jobs.	VMEMACCM_NO_JOB
			VMEMACCM_WRITEJOB
			VMEMACCM_READJOB
			VMEMACCM_READBLANKJOB
			VMEMACCM_COMPAREJOB
			VMEMACCM_ERASEJOB
			VMEMACCM_VMEMJOB
vMemAccM_VariantIdentifierType	uint8	Type for unique numeric address area memory mapping variant identifier.	VMEMACCM_ISBLANKJOB
			0-255

Table 4-1 Type definitions

vMemAccM_MemoryInfoType

This structure contains information on memory device characteristics and can be accessed via the `vMemAccM_GetMemoryInfo()` service. The elements of `vMemAccM_MemoryInfoType` are described in the table below.

Struct Element Name	C-Type	Description	Value Range
VirtualStartAddress	uint32	Virtual start address of sub address area	0 – 4294967295
PhysicalStartAddresses	uint32	Physical start address of sub address area	0 – 4294967295
MaxOffset	uint32	The maximum offset from both start addresses - virtual and physical. (== sub address area length - 1)	0 – 4294967295
NumberOfSectors	uint32	Number of sectors of sub address area	0 – 4294967295
SectorSize	uint32	Size of a sector in bytes	0 – 4294967295
SectorBurstSize	uint32	Size of a sector burst in bytes. Equals SectorSize in case burst is disabled.	0 – 4294967295
PageSize	uint32	Size of a page in bytes	0 – 4294967295
PageBurstSize	uint32	Size of a page burst in bytes. Equals PageSize in case burst is disabled.	0 – 4294967295
PageBurstRamAlignment	uint32	Alignment for page burst RAM, 1 if burst disabled.	0 – 4294967295
HwId	enum: vMemAccM_HwIdType	Referenced vMem hardware identifier	0 – 4294967295

Table 4-2 vMemAccM_MemoryInfoType

vMemAccM_StateType

This structure contains information the current processing state of the vMemAccM.

Struct Element Name	C-Type	Description	Value Range
Address	uint32	Address of the currently active address area request.	0 – 4294967295
Length	uint32	Length of the currently active address area request.	0 – 4294967295
vMemInstanceId	uint32	Instance ID of the current vMem request.	0 – 4294967295
vMemAddress	uint32	Physical address of the current vMem request.	0 – 4294967295
vMemLength	uint32	Length of the current vMem request.	0 – 4294967295
CurrentJob	vMemAccM_JobType	The job vMemAccM processes for the active address area.	-

Struct Element Name	C-Type	Description	Value Range
vMemResult	vMemAccM_JobResultType	Current or last vMem result.	-

Table 4-3 vMemAccM_StateType

4.2 Services provided by vMemAccM

4.2.1 vMemAccM_InitMemory

Prototype	
void vMemAccM_InitMemory (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Function for *_INIT_*-variable initialization. Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	
Particularities and Limitations	
-	
Call context	
> TASK (usually start-up sequence) > This function is synchronous. > This function is non-reentrant.	

Table 4-4 vMemAccM_InitMemory

4.2.2 vMemAccM_Init

Prototype	
void vMemAccM_Init (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initialization function. This function initializes the module vMemAccM. It initializes all variables and sets the module state to initialized. All configured vMems will also be initialized.	

Particularities and Limitations

- > `vMemAccM_InitMemory` has been called unless `vMemAccM_ModuleInitialized` is initialized by start-up code.

Call context

- > TASK (usually ECU initialization sequence e.g. EcuM driver Init-phases)
- > This function is synchronous.
- > This function is non-reentrant.

Table 4-5 `vMemAccM_Init`

4.2.3 vMemAccM_Read

Prototype	
<code>Std_ReturnType vMemAccM_Read (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType SourceAddress, vMemAccM_DataPtrType DestinationPtr, vMemAccM_LengthType Length)</code>	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
SourceAddress [in]	Virtual address within address area from which the read job shall start.
DestinationPtr [out]	Pointer to where the read data shall be stored.
Length [in]	Number of bytes that shall be read.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Reads NV memory content of the related address area to passed buffer – from address with length number of bytes. Queues the job, which will be processed during vMemAccM_MainFunction. Its status can be polled via vMemAccM_GetJobResult.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is asynchronous. > This function is reentrant.	

Table 4-6 vMemAccM_Read

4.2.4 vMemAccM_ReadBlank

Prototype	
<code>Std_ReturnType vMemAccM_ReadBlank (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType SourceAddress, vMemAccM_DataPtrType DestinationPtr, vMemAccM_LengthType Length)</code>	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
SourceAddress [in]	Virtual address within address area from which the read blank job shall start.
DestinationPtr [out]	Pointer to where the read data shall be stored.
Length [in]	Number of bytes that shall be read.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise

Functional Description
Reads NV memory content of the related address area to passed buffer – from address with length number of bytes. Before accessing the NV RAM, a blank check will be performed to avoid accessing erased memory. Should a blank page be found, the buffer will be filled with an erased value at the respective location instead. The erase value is a constant value of 0xFF. Queues the job, which will be processed during <code>vMemAccM_MainFunction</code>. Its status can be polled via <code>vMemAccM_GetJobResult</code>.
Particularities and Limitations
-
Call context
> TASK > This function is asynchronous. > This function is reentrant.

Table 4-7 vMemAccM_ReadBlank

4.2.5 vMemAccM_Write

Prototype	
Std_ReturnType vMemAccM_Write (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_ConstDataPtrType SourcePtr, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
TargetAddress [in]	Virtual address within address area from which write job shall start.
SourcePtr [in]	Pointer to the data which shall be written to memory.
Length [in]	Number of bytes that shall be written. Keep in mind, that the memory is only page-wise writeable. Hence requested length must always be aligned to page size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Writes the passed data to NV memory of the related address area – from address with length number of bytes. Queues the job, which will be processed during <code>vMemAccM_MainFunction</code>. Its status can be polled via <code>vMemAccM_GetJobResult</code>. If enabled for write jobs, vMemAccM uses burst capabilities of underlying vMem.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is asynchronous.	

> This function is reentrant.

Table 4-8 vMemAccM_Write

4.2.6 vMemAccM_Erase

Prototype	
Std_ReturnType vMemAccM_Erase (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
TargetAddress [in]	Virtual address within address area from which erase job shall start.
Length [in]	Number of bytes that shall be erased. Keep in mind, that the memory is only sector-wise erasable. Hence requested length must always be aligned to sector size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Erases NV memory content of the related address area – from requested address with length number of bytes. Queues the job, which will be processed during vMemAccM_MainFunction. Its status can be polled via vMemAccM_GetJobResult. If enabled for write jobs, vMemAccM uses burst capabilities of underlying vMem.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is asynchronous. > This function is reentrant.	

Table 4-9 vMemAccM_Erase

4.2.7 vMemAccM_Compare

Prototype	
Std_ReturnType vMemAccM_Compare (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType SourceAddress, vMemAccM_ConstDataPtrType DataPtr, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
SourceAddress [in]	Virtual address within address area from which compare job shall start.
DataPtr [in]	Pointer to user data which shall be compared to data in NV memory.
Length [in]	Number of bytes that shall be compared.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Compares the passed data to NV memory content of related address area – from address with length number of bytes. The job will be finished immediately, if all bytes match or even if the first not equal byte was detected. Queues the job, which will be processed during vMemAccM_MainFunction. Its status can be polled via vMemAccM_GetJobResult.	
Particularities and Limitations	
VMEMACCM_COMPARE_API == STD_ON	
Call context	
> TASK > This function is asynchronous. > This function is reentrant.	

Table 4-10 vMemAccM_Compare

4.2.8 vMemAccM_IsBlank

Prototype	
Std_ReturnType vMemAccM_IsBlank (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area. Used to access the virtual information.
TargetAddress [in]	Virtual address within address area pointing to the first byte of the request.
Length [in]	Number of bytes that shall be checked. Value is hardware dependent and must be aligned to the smallest checkable size, usually a page size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise

Functional Description
Checks the passed address space to be blank, i.e. erased and writable. Queues the job, which will be processed during <code>vMemAccM_MainFunction</code> . Its status can be polled via <code>vMemAccM_GetJobResult</code> .
Particularities and Limitations
-
Call context
> TASK > This function is asynchronous. > This function is reentrant.

Table 4-11 vMemAccM_IsBlank

4.2.9 vMemAccM_HwSpecificReq

Prototype	
<pre>Std_ReturnType vMemAccM_HwSpecificReq (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_HwIdType HwId, vMemAccM_vMemHwSpecificFuncIndexType vMemHwSpecificFuncIndex, vMemAccM_DataPtrType DataPtr, vMemAccM_LengthType Length)</pre>	
Parameter	
AddressAreaId [in]	AddressAreaId as array index, used to access the virtual information.
HwId [in]	HwId pointing to the hardware driver the job shall be forwarded to.
vMemHwSpecificFuncIndex [in]	MemHwSpecificFuncIndex as an array index pointing to the hardware specific function pointer.
DataPtr [in,out]	DataPtr pointing to the job buffer. Value can be NULL_PTR, if not needed (DataSize == 0). If != NULL_PTR, it must stay valid until the AddressAreaId is idle (interface between vMem and upper layer, vMemAccM does not use this pointer).
Length [in]	DataSize of the data in bytes. Value can be 0, if DataPtr is NULL_PTR.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise.
Functional Description	
Sets up a hardware specific request. vMemAccM handles the requests in the same way as any other asynchronous request, except it does not know anything about the invoked vMem function – vMemAccM just invokes one vMem specific function indexed by the passed vMemFuncIndex from provided function pointer table. Queues the job, which will be processed during <code>vMemAccM_MainFunction</code> . Its status can be polled via <code>vMemAccM_GetJobResult</code> .	
Particularities and Limitations	
See <code>vMemAccM_vMemApi.h</code> for more information on how to implement and use vMem specific requests. A precondition is that vMem provides the requested functionality.	

Call context
> TASK
> This function is asynchronous.
> This function is reentrant.

Table 4-12 vMemAccM_HwSpecificReq

4.2.10 vMemAccM_MainFunction

Prototype	
void vMemAccM_MainFunction (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Takes care about job processing. Also takes care of scheduling underlying vMems.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is non-reentrant.	

Table 4-13 vMemAccM_MainFunction

4.2.11 vMemAccM_Cancel

Prototype	
void vMemAccM_Cancel (vMemAccM_AddressAreaIdType AddressAreaId)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Return code	
void	none
Functional Description	
Cancels the pending job for the address area related to passed address area ID. Canceling effects only address areas in pending state, otherwise the request will be ignored.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous.	

> This function is non-reentrant.

Table 4-14 vMemAccM_Cancel

4.2.12 vMemAccM_GetJobResult

Prototype	
vMemAccM_JobResultType vMemAccM_GetJobResult (vMemAccM_AddressAreaIdType AddressAreaId)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Return code	
vMemAccM_JobResultType	Most recent job result of the passed address area ID.
Functional Description	
Returns the job result of the address area related to passed address area ID.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-15 vMemAccM_GetJobResult

4.2.13 vMemAccM_GetMemoryInfo

Prototype	
Std_ReturnType vMemAccM_GetMemoryInfo (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType Address, vMemAccM_LengthType Length, vMemAccM_MemoryInfoPtrType MemoryInfoPtr)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Address [in]	Virtual address within address area from which corresponding memory device information shall be retrieved.
Length [in]	Number of bytes which shall be covered.
MemoryInfoPtr [out]	Pointer to where the memory device information shall be stored.
Return code	
Std_ReturnType	E_OK job accepted, E_NOT_OK job not accepted, or the passed address space is not continuous.
Functional Description	
Writes memory device information related to passed address area ID and address range to the passed pointer. GetMemoryInfo checks the passed address and length to cover a continuous memory, otherwise the request will be rejected. A detailed description of the retrieved information can be found in Table 4-2 vMemAccM_MemoryInfoType If the address range/ space covers more than one sub address area, vMemAccM will write the memory information of the first sub address area. The user has to increase his address to get the memory information of the next sub address area and so on.	

Particularities and Limitations
-
Call context
> TASK
> This function is synchronous.
> This function is reentrant.

Table 4-16 vMemAccM_GetMemoryInfo

4.2.14 vMemAccM_SetReadOnlyMode

Prototype	
Std_ReturnType vMemAccM_SetReadOnlyMode (vMemAccM_AddressAreaIdType AddressAreaId, boolean ReadOnly)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
ReadOnly [in]	TRUE -> address area is set to read only FALSE -> address area is set to writable
Return code	
Std_ReturnType	E_OK request accepted, address area is read only, E_NOT_OK otherwise
Functional Description	
Sets the address area related to passed address area ID to read only or writable. ReadOnly == TRUE: vMemAccM will only accept read jobs for the related address area, other jobs will be rejected. ReadOnly == FALSE: vMemAccM will accept all jobs. This is the default value after initialization.	
Particularities and Limitations	
-	
Call context	
> TASK	
> This function is synchronous.	
> This function is reentrant.	

Table 4-17 vMemAccM_SetReadOnlyMode

4.2.15 vMemAccM_GetProcessedLength

Prototype	
Std_ReturnType vMemAccM_GetProcessedLength (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_LengthPtrType LengthPtr)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
LengthPtr [out]	Accumulated number of bytes that have already been processed of most recent job.

Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise
Functional Description	
Returns the number of bytes processed during the most recent job for passed address area ID (for more information see chapter 2.9).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-18 vMemAccM_GetProcessedLength

4.2.16 vMemAccM_GetState

Prototype	
Std_ReturnType vMemAccM_GetState (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_StatePtrType StatePtr)	
Parameter	
AddressAreaId[in]	AddressArea Identifier of the address area to be checked.
StatePtr [out]	Pointer to a structure where vMemAccM writes the state information in.
Return code	
Std_ReturnType	E_OK provided structure was filled with state information; E_NOT_OK otherwise
Functional Description	
Returns the current processing state of the given AddressArea. The following states are possible: - AddressArea idle/job still within the queue (not processing a job): <code>StatePtr->CurrentJob == VMEMACCM_NO_JOB</code> - AddressArea busy internal (preparing or finalizing a job, vMem is idle): <code>StatePtr->CurrentJob != VMEMACCM_NO_JOB</code> <code>&& StatePtr->vMemResult != VMEMACCM_JOB_PENDING</code> - AddressArea busy, vMem busy (vMemAccM waits for the vMem to process its job): <code>StatePtr->CurrentJob != VMEMACCM_NO_JOB</code> <code>&& StatePtr->vMemResult == VMEMACCM_JOB_PENDING</code> vMemAccM provides the job type, virtual address space, vMem result and the physical address space information of the address area. Depending on the current state of the vMem solution (vMemAccM idle/busy, vMem idle/busy) more information is passed through the given StatePtr to narrow the information down. This service shall be used, if deeper vMemAccM processing information is necessary than provided by the GetJobResult API. For example in case the user has to know if and which NV RAM device is currently in use.	
Particularities and Limitations	
The given StatePtr will be completely overwritten within this function.	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-19 vMemAccM_GetState

4.2.17 vMemAccM_SetActiveVariant

Prototype	
Std_ReturnType vMemAccM_SetActiveVariant (vMemAccM_VariantIdType VariantId)	
Parameter	
VariantId [in]	Variant identifier to be set as the active variant.
Return code	
Std_ReturnType	E_OK Variant was set internally; E_NOT_OK otherwise
Functional Description	
Sets the active memory mapping variant to the passed value. The setting is synchronous, that means after the function returns, the variant is set and all following requests to an address area using the variant feature, the new variant will be used to forward the request to underlying module(s). See chapter 2.6.1 for more information about the memory mapping variants.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-20 vMemAccM_SetActiveVariant

4.2.18 vMemAccM_ErrorNotification

Prototype	
Std_ReturnType vMemAccM_ErrorNotification (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType Address, vMemAccM_ErrorNotifResultType Result)	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Address [in]	Virtual address causing the user to invoke the function.
Result [in]	Result to be combined with the actual vMem job result.
Return code	
Std_ReturnType	E_OK Result set internally; E_NOT_OK otherwise.
Functional Description	
Stores the passed result and combines it with the actual vMem job result, in case the passed AddressAreaId and the Address matches the currently active job. All other requests will be rejected. See chapter 2.5 for more information about the usage of this API.	

Particularities and Limitations
-
Call context
> ANY
> This function is synchronous.
> This function is reentrant.

Table 4-21 vMemAccM_ErrorNotification

4.2.19 vMemAccM_GetVersionInfo

Prototype	
void vMemAccM_GetVersionInfo (Std_VersionInfoType *VersionInfo)	
Parameter	
VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information. vMemAccM_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
VMEMACCM_VERSION_INFO_API == STD_ON	
Call context	
> TASK	
> This function is synchronous.	
> This function is reentrant.	

Table 4-22 vMemAccM_GetVersionInfo

4.2.20 vMemAccM_RejectNvRamRequests

Prototype	
Std_ReturnType vMemAccM_RejectNvRamRequests (boolean RejectRequests)	
Parameter	
RejectRequests [in]	TRUE -> Future requests shall be rejected. FALSE -> Future requests shall be accepted.
Return code	
Std_ReturnType	E_OK Requests will be handled according to the given value E_NOT_OK Request was not accepted
Functional Description	
Makes the vMemAccM reject/allow all requests which access the NV RAM. RejectRequests == TRUE: vMemAccM will reject all jobs. RejectRequests == FALSE: vMemAccM will accept all jobs. The default is to accept all NV RAM requests. The service will not accept the request if there is a pending job in the queue. Make sure the vMemAccM is not pending before calling this service.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-23 vMemAccM_RejectNvRamRequests

4.2.21 vMemAccM_ActivatevMem

Prototype	
Std_ReturnType vMemAccM_ActivatevMem (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType Address, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of the address area.
Address [in]	Address of the first byte of the vMem.
Length [in]	Address range of the vMem.
Return code	
Std_ReturnType	E_OK Activation was successful E_NOT_OK Activation failed

Functional Description
Allows the activation of an alternate vMem. Requests will be routed through the alternate vMem instead of the corresponding pre-configured vMem. Activation will not work if the internal validation of the given vMem fails.
Particularities and Limitations
Cannot be used if configured as a multi binary satellite.
Call context
> TASK > This function is synchronous. > This function is non-reentrant.

Table 4-24 vMemAccM_ActivatevMem

4.2.22 vMemAccM_DeactivatevMem

Prototype	
Std_ReturnType vMemAccM_DeactivatevMem (vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType Address, vMemAccM_LengthType Length)	
Parameter	
AddressAreaId [in]	Numeric identifier of the address area.
Address [in]	Address of the first byte of the vMem.
Length [in]	Address range of the vMem.
Return code	
Std_ReturnType	E_OK Deactivation was successful E_NOT_OK Deactivation failed
Functional Description	
Allows the deactivation of an alternate vMem. It will no longer be used for memory requests, instead the pre-configured vMem will be used.	
Particularities and Limitations	
Cannot be used if configured as a multi binary satellite.	
Call context	
> TASK > This function is synchronous. > This function is non-reentrant.	

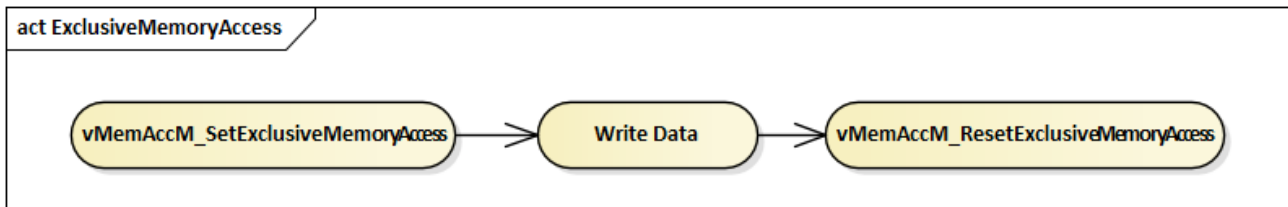
Table 4-25 vMemAccM_DeactivatevMem

4.2.23 vMemAccM Exclusive Memory Access

vMemAccM provides the possibility to configure more than one AddressArea. An address area can be configured for the User Memory Stack for example, a second address area for the User Software Download Stack.

The vMemAccM offers APIs to set and reset exclusive memory access for a specific address area. However, exclusive memory access should only be used in special use cases because processing of all other address area's is halted.

To realize an exclusive memory access the function pair `vMemAccM_SetExclusiveMemoryAccess` and `vMemAccM_ResetExclusiveMemoryAccess` shall be used. In case an exclusive memory access is set for an address area, it shall also be reset after execution of writing data for example crash data or performing a `WriteAll()`.

**Caution**

Use the Exclusive Memory Access feature with care, it blocks all other memory accesses until the reset API call. The feature does not consider Address Area priorities!

Use case: fast- and short-term memory access.

4.2.23.1 vMemAccM_SetExclusiveMemoryAccess

Prototype	
<code>Std_ReturnType vMemAccM_SetExclusiveMemoryAccess (vMemAccM_AddressAreaIdType AddressAreaId)</code>	
Parameter	
AddressAreaId [in]	Numeric identifier of the address area.
Return code	
Std_ReturnType	E_OK Setting of Exclusive Memory Access was successful E_NOT_OK Setting of Exclusive Memory Access failed
Functional Description	
Sets the exclusive access to the memory for an AddressArea. In case the exclusive memory access is set for a specific AddressArea: > Only jobs of this specific AddressArea are processed. > The jobs for all other AddressAreas are accepted but they are not processed and remain pending. > Currently processed jobs for all other AddressAreas will be paused. A currently active job step within vMem/hardware will be finished, but the job is not processed further.	
Particularities and Limitations	
-	
Call context	

- > TASK
- > This function is synchronous.
- > This function is reentrant.

Table 4-26 vMemAccM_SetExclusiveMemoryAccess

4.2.23.2 vMemAccM_ResetExclusiveMemoryAccess

Prototype	
void vMemAccM_ResetExclusiveMemoryAccess (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Resets the exclusive memory access of the AddressArea set by vMemAccM_SetExclusiveMemoryAccess().	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 4-27 vMemAccM_ResetExclusiveMemoryAccess

4.3 Services used by vMemAccM

In the following table services provided by other components, which are used by the vMemAccM are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET (Development Error Tracer)	Det_ReportError
vMem	vMem_Init vMem_Read vMem_Write vMem_Erase vMem_IsBlank vMem_GetJobResult vMem_<HwSpecificReq> vMem_MainFunction

Component	API
SchM (Schedule Manager)	SchM_Enter_vMemAccM_VMEMACCM_EXCLUSIVE_AREA_0 SchM_Exit_vMemAccM_VMEMACCM_EXCLUSIVE_AREA_0

Table 4-28 Services used by the vMemAccM

4.4 Callback Functions

The vMemAccM does not implement any callback functions.

4.5 Configurable Interfaces

4.5.1 Notifications

At its configurable interfaces the vMemAccM defines notifications that can be mapped to callback functions provided by other modules. The mapping is not statically defined by the vMemAccM but can be performed at configuration time. The function prototypes that can be used for the configuration have to match the appropriate function prototype signatures, which are described in the following sub-chapters.

4.5.1.1 JobEndNotification (configurable name)

Prototype	
<pre>void JobEndNotification(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_JobResultType JobResult)</pre>	
Parameter	
AddressAreaId [in]	Id of the address area where the job was completed.
JobResult [in]	Result of the job.
Return code	
-	-
Functional Description	
Notification, which is called upon job completion of an address area. The notification will be called regardless of the job result. The vMemAccM is ready to accept new jobs within this notification.	
Particularities and Limitations	
> The function will only be called once, regardless of job retries which might be necessary due to reasons in the lower layer. > Canceling an ongoing job will also result in a notification. > Function must be declared within the header file whose file name is also configurable.	
Call context	
> TASK > Called upon job completion	

Table 4-29 JobEndNotification

4.5.2 Callout Functions

The vMemAccM does not define any callout functions.

5 Configuration

In the vMemAccM the attributes can be configured according to/ with the following methods/ tools:

- > DaVinci Configurator 5 (recommended)
- > AUTOSAR Generic Configuration Editor (GCE)

5.1 Configuration Variants

The vMemAccM supports the configuration variants

- > VARIANT-PRE-COMPILE

5.2 Configuration with DaVinci Configurator

The configuration parameters are generally well described within the tool and the vMemAccM_bswmd.arxml file. However, there are some more complex dependencies between certain configuration parameters which are described in more detail in the following.

5.2.1 Read Request Length

The maximum number of bytes the vMemAccM requests within one vMem read request can be configured for each address area within the parameter `vMemAccMReadRequestLength`.

If `vMemAccMCompareApi` is enabled, the configured values will also be used to calculate the internal buffer size – see chapter 2.12. Therefore, it is recommended to choose the read request length with care, since it determines the internal RAM usage.

5.2.2 Memory Address Mapping

To achieve a correct mapping from virtual to physical addresses the sub address areas must be configured according to the following instructions:

5.2.2.1 Physical Memory

- > The sector batch where the physical memory of the related sub address area is located must be referenced in the parameter **Sector Ref**. All memory device specific information, such as page and sector size, are to be configured within the vMem in the sector batch.
- > The physical memory of any sub address area must lie within the range of the referenced sector batch. A validator checks this for every sub address area. The following explanations shall only help to understand what the validation rule checks. The range of a sector batch is determined by its start and end address. The start address of the sector batch has to be configured in the vMem parameter **Start Address**. The end address of the sector batch can be computed using the vMem parameter **Number Of Sectors** and **Sector Size** using the formula

$$\text{EndAddress} = \text{StartAddress} + \text{NumberOfSectors} * \text{SectorSize} - 1$$

The physical start address of a sub address area is configured within the vMemAccM parameter **Physical Start Address**. The physical end address of the sub address

area can be computed in the same way as the end address of the sector batch, but with the vMemAccM parameter **Number Of Sectors**. The validation rule checks that both the physical start address and physical end address lie within the range of the referenced sector batch.

- > The same sector batch can be referenced from multiple sub address areas from different address areas. The physical memory of the sub address areas referencing the same sector batch must NOT overlap within this sector batch. This is also checked by a validator.
- > The physical start address of any sub address area must be aligned. That is, it must be a multiple of the sector size of the referenced sector batch. This is also checked by a validator.

5.2.2.2 Virtual Memory

- > The sub address areas must not overlap within the address area. A validator checks this for every address area. The following explanations shall only help to understand what the validation rule checks. The virtual start address of a sub address area is configured in the parameter **Virtual Start Address**. The virtual end address of a sub address area can be calculated using the formula

$$\text{VirtualEndAddress} = \text{VirtualStartAddress} + \text{NumberOfSectors} * \text{SectorSize} - 1$$

where NumberOfSectors is the number of sectors of the sub address. The validator checks, that for any pair of consecutive sub address areas, the virtual end address of the first sub address area is smaller than the virtual start address of the second sub address area.

- > The virtual end address of a sub address area must not exceed the value 4294956295 or 0xFFFF FFFF in hexadecimal, since the type used within the implementation is uint32. This is also checked by a validator.

5.2.3 Parallel processing

Configuration of synchronization groups must be done via the optional parameter `"/MICROSAR/vMemAccM/vMemAccMSynchronizationGroupConfiguration"`.

When configuring synchronization groups, at least two instances of the parameter must be created. Otherwise, one synchronization group will exist, which all sub address areas will be part of.

The parameter holds references to sub address areas. All sub address areas referenced by a synchronization group will be processed sequentially within that group. When using synchronization groups, **all** sub address areas must be referenced by exactly one synchronization group instance.

Beware that jobs cannot be parallelized within a single address area, even if multiple sub address areas of that address area are in different synchronization groups.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
Address area	Virtual area of memory, not necessarily continuously filled. Asynchronous jobs of vMemAccM are requested for an address area.
Page	Physical area of memory, smallest writable unit.
Page Burst / Write Burst	A page burst can be used for improved performance of write jobs. Instead of writing one page at a time, multiple pages within the same sector batch can be written at once.
Pseudo-physical address	The pseudo-physical addresses refer to the addresses which are provided by the memory driver. They are not the physical addresses used by the microprocessor.
Sector	Physical area of memory, smallest erasable unit.
Sector Burst / Erase Burst	A sector burst can be used for improved performance of erase jobs. Instead of erasing one sector at a time, multiple sectors within the same sector batch can be erased at once.
Sub address area	Part of an address area, which corresponds to a physical continuous area of memory.

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
EcuM	ECU Manager
Fee	Flash EEPROM Emulation
HSM	Hardware Security Module
ISR	Interrupt Service Routine
MemIf	Memory Abstraction Interface
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NV, NVRAM	Non Volatile Random Access Memory
NvM	NVRAM Manager
OTA	Over The Air

Fls_30_vMemAccM	Adapter between AUTOSAR memory stack (Fee) and Software Download stack (vMemAccM) Note: previously this component was called vFlsAcc
vMem	Vector Memory (Driver)
vMemAccM	Vector Memory Access Manager
vSwUpdM	Vector Software Update Manager

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com